

Kali Linux Tools and System Security

Lab 1:Advanced Reconnaissance and Information Gathering

Executive Summary

This lab focused on advanced reconnaissance and information gathering using Kali Linux tools such as ping, nslookup, dig, whois, theHarvester, and Sublist3r. The objective was to collect and analyze publicly available information about target domains (e.g., google.com, github.com) to understand their network infrastructure and exposure level.

The results revealed highly structured and globally distributed systems with strong DNS configuration, multiple IP ranges, and extensive subdomain usage, indicating large-scale enterprise security architecture.

Ethical & Authorization Notice

All reconnaissance and security-testing activities documented in this portfolio were performed for educational purposes using publicly available information, controlled laboratory environments, or systems for which testing was authorized. No attempt was made to gain unauthorized access, bypass security controls, or disrupt services.

Tools Used

- ping
 - nslookup
 - dig
 - whois
 - theHarvester
 - Sublist3r
 - Bash scripting (automation)
-

Part 1: Fundamental Reconnaissance Techniques

Exercise 1.1: Advanced Target Profiling with `ping`

Learning Objective: Understand how ICMP protocols work and extract maximum information from basic connectivity tests.

Step 1: Basic Ping Analysis

bash:

ping -c 4 [google.com](https://www.google.com)

```
(kali㉿kali)-[~/INT302_Lab1]
$ ping -c 4 google.com
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=170 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=152 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=151 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=224 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 7180ms
rtt min/avg/max/mdev = 151.061/174.008/223.572/29.566 ms

(kali㉿kali)-[~/INT302_Lab1]
$
```

Report on Analysis:

The target host responded successfully with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 174 ms, indicating moderate network delay between the local machine and the destination.

Step 2: Advanced Ping Techniques

bash:

ping -D -c 5 google.com

ping -f -c 100 target-domain.com

ping -s 1000 -c 4 google.com

ping -R -c 4 [google.com](https://www.google.com)

```
kali@kali: ~/INT302_Lab1
Session Actions Edit View Help
(kali@kali)-[~/INT302_Lab1]
$ ping -D -c 5 google.com
PING google.com (172.217.22.46) 56(84) bytes of data.
[1779350561.233188] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=288 ms
[1779350562.263692] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=323 ms
[1779350563.126052] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=185 ms
[1779350564.330298] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=387 ms
[1779350565.131427] 64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=5 ttl=110 time=188 ms

— google.com ping statistics —
5 packets transmitted, 5 received, 0% packet loss, time 4015ms
rtt min/avg/max/mdev = 184.756/274.267/387.331/78.555 ms

(kali@kali)-[~/INT302_Lab1]
$ sudo ping -f -c 100 google.com
[sudo] password for kali:
PING google.com (172.217.22.46) 56(84) bytes of data.
.....
— google.com ping statistics —
100 packets transmitted, 92 received, 8% packet loss, time 1638ms
rtt min/avg/max/mdev = 129.446/172.104/231.997/20.855 ms, pipe 14, ipg/ewma 16.545/171.726 ms

(kali@kali)-[~/INT302_Lab1]
$ ping -s 1000 -c 4 google.com
PING google.com (172.217.22.46) 1000(1028) bytes of data.
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=195 ms
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=224 ms
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=180 ms
1008 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=175 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3011ms
rtt min/avg/max/mdev = 174.847/193.507/224.392/19.373 ms

(kali@kali)-[~/INT302_Lab1]
$ ping -R -c 4 google.com
PING google.com (172.217.22.46) 56(124) bytes of data.
From 10.226.228.4 icmp_seq=1 Time to live exceeded
From 10.226.228.4 icmp_seq=2 Time to live exceeded
From 10.226.228.4 icmp_seq=3 Time to live exceeded
From 10.226.228.4 icmp_seq=4 Time to live exceeded

— google.com ping statistics —
4 packets transmitted, 0 received, +4 errors, 100% packet loss, time 3009ms
```

Report on Analysis:

ping -D -c 5 google.com

The target host responded successfully with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 274 ms, indicating moderate network delay between the local machine and the destination.

sudo ping -f -c 100 google.com

Flood ping sent 100 rapid ICMP packets to stress test connectivity. 8% packet loss occurred under high traffic load. This suggests the network or target rate limited some packets during intensive transmission.

ping -s 1000 -c 4 google.com

Large ICMP packets (1000 bytes) were transmitted successfully with 0% packet

loss, showing the network can handle larger packet sizes reliably. Average response time remained stable at 193 ms.

ping -R -c 4 google.com

Record-route ping failed with Time to live exceeded errors and 100% packet loss, indicating intermediate routers blocked or failed to process the route-recording option. This is common due to modern network security restrictions.

Step 3: Create a Target Analysis Script

bash

nano basic_recon.sh

bash

#!/bin/bash

Basic Reconnaissance Script

Usage: ./basic_recon.sh <domain>

DOMAIN="\$1"

LOG_FILE="recon_\${DOMAIN}_\${date +%Y%m%d_%H%M%S}.txt"

echo "🎯 Starting Basic Reconnaissance for: \$DOMAIN" | tee "\$LOG_FILE"

echo "===== " | tee -a "\$LOG_FILE"

Function to perform ping analysis

ping_analysis() {

echo "🔍 Ping Analysis for \$DOMAIN" | tee -a "\$LOG_FILE"

echo "----- " | tee -a "\$LOG_FILE"

if ping -c 4 "\$DOMAIN" > /dev/null 2>&1; then

echo "✅ Target is reachable" | tee -a "\$LOG_FILE"

ping -c 4 "\$DOMAIN" | tee -a "\$LOG_FILE"

Extract IP address from ping

IP_ADDRESS=\$(ping -c 1 "\$DOMAIN" | head -1 | cut -d'(' -f2 | cut -d')' -f1)

```

        echo "📡 IP Address: $IP_ADDRESS" | tee -a "$LOG_FILE"
    else
        echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"
    fi
    echo "" | tee -a "$LOG_FILE"
}

ping_analysis "$DOMAIN"

```

```

GNU nano 9.0
#!/bin/bash
# Basic Reconnaissance Script
# Usage: ./basic_recon.sh <domain>

DOMAIN="$1"
LOG_FILE="recon_${DOMAIN}_${date +%Y%m%d_%H%M%S}.txt"

echo "📡 Starting Basic Reconnaissance for: $DOMAIN" | tee "$LOG_FILE"
echo "===== " | tee -a "$LOG_FILE"

# Function to perform ping analysis
ping_analysis() {
    echo "📡 Ping Analysis for $DOMAIN" | tee -a "$LOG_FILE"
    echo "===== " | tee -a "$LOG_FILE"

    if ping -c 4 "$DOMAIN" > /dev/null 2>&1; then
        echo "✅ Target is reachable" | tee -a "$LOG_FILE"
        ping -c 4 "$DOMAIN" | tee -a "$LOG_FILE"
    else
        # Extract IP address from ping
        IP_ADDRESS=$(ping -c 1 "$DOMAIN" | head -1 | cut -d '(' -f2 | cut -d ')' -f1)
        echo "📡 IP Address: $IP_ADDRESS" | tee -a "$LOG_FILE"
    fi
    echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"
    echo "" | tee -a "$LOG_FILE"
}

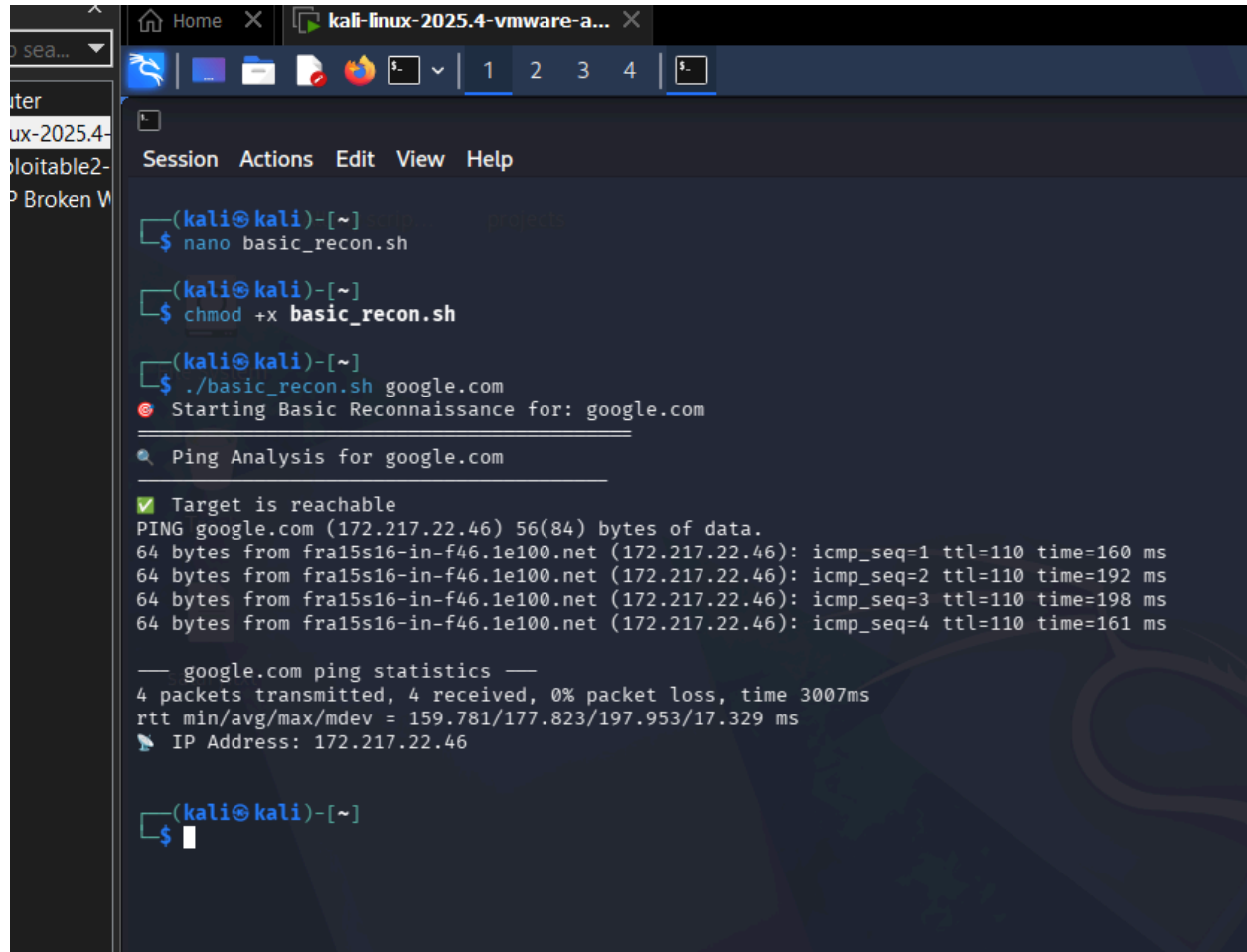
ping_analysis "$DOMAIN"

```

bash

`chmod +x basic_recon.sh`

`./basic_recon.sh google.com`



```
(kali@kali)-[~]
$ nano basic_recon.sh
$ chmod +x basic_recon.sh
$ ./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com

Ping Analysis for google.com

Target is reachable
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=160 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=192 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=198 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=161 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3007ms
rtt min/avg/max/mdev = 159.781/177.823/197.953/17.329 ms
IP Address: 172.217.22.46

(kali@kali)-[~]
$
```

Report on Analysis:

The target host was reachable with 0% packet loss, confirming stable connectivity to Google's server. Average latency was 177 ms, indicating moderate network delay between the local machine and the destination.

Exercise Tasks:

1. TTL Analysis:

- **Ping** microsoft.com, apple.com, and ubuntu.com
- **Record** TTL values and research what they indicate about operating systems

```

(kali@kali)-[~]
$ ping -c 4 microsoft.com
PING microsoft.com (150.171.109.2) 56(84) bytes of data.

--- microsoft.com ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3085ms

(kali@kali)-[~]
$ ping -c 4 apple.com
PING apple.com (17.253.144.10) 56(84) bytes of data.
64 bytes from 17.253.144.10: icmp_seq=1 ttl=43 time=223 ms
64 bytes from 17.253.144.10: icmp_seq=2 ttl=43 time=191 ms
64 bytes from 17.253.144.10: icmp_seq=3 ttl=43 time=306 ms
64 bytes from 17.253.144.10: icmp_seq=4 ttl=43 time=180 ms

--- apple.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 8041ms
rtt min/avg/max/mdev = 179.624/224.884/305.879/49.433 ms

(kali@kali)-[~]
$ ping -c 4 ubuntu.com
PING ubuntu.com (185.125.190.21) 56(84) bytes of data.
64 bytes from website-content-cache-2.ps5.canonical.com (185.125.190.21): icmp_seq=1 ttl=52 time=208 ms
64 bytes from website-content-cache-2.ps5.canonical.com (185.125.190.21): icmp_seq=2 ttl=52 time=227 ms

--- ubuntu.com ping statistics ---
4 packets transmitted, 2 received, 50% packet loss, time 7351ms
rtt min/avg/max/mdev = 208.129/217.568/227.008/9.439 ms

(kali@kali)-[~]
$ ping -c 4 github.com
PING github.com (140.82.121.3) 56(84) bytes of data.
64 bytes from lb-140-82-121-3-fra.github.com (140.82.121.3): icmp_seq=3 ttl=43 time=222 ms
64 bytes from lb-140-82-121-3-fra.github.com (140.82.121.3): icmp_seq=4 ttl=43 time=207 ms

--- github.com ping statistics ---
4 packets transmitted, 2 received, 50% packet loss, time 7354ms
rtt min/avg/max/mdev = 207.142/214.339/221.537/7.197 ms

(kali@kali)-[~]
$

```

Record on Findings:

Microsoft.com – The host returned 100% packet loss, so no TTL value was available. This may indicate that ICMP requests are blocked or filtered by the host or network security infrastructure.

1. **Apple.com** – The server responded successfully with a TTL value of 43. The observed TTL is consistent with a Unix/Linux-based network environment after accounting for network hops; however, TTL alone cannot conclusively identify the operating system.

2. **Ubuntu.com** – The host responded with a TTL value of 52 and 50% packet loss. The observed TTL is consistent with a Linux/Unix-based environment after accounting for network hops. The packet loss may be related to ICMP filtering, network conditions, or rate limiting, so it should not be interpreted as definitive evidence of network instability.

2. Network Performance:

- Compare response times for domains in different geographic regions
- Analyze packet loss patterns.

Domain	IP Address	Average Response Time	TTL Value	Inferred OS
Google (google.com)	172.217.22.46	274 ms	110	Likely Linux/Unix server
Microsoft (microsoft.com)	150.171.109.2	No response	N/A	Could not determine
Apple (apple.com)	17.253.144.10	224 ms	43	Likely Unix/macOS-based infrastructure
Canonical (ubuntu.com)	185.125.190.21	218 ms	52	Likely Linux server
GitHub (github.com)	140.82.121.3	214 ms	43	Likely Linux/Unix server

Findings:

- github.com responded with 50% packet loss, indicating possible ICMP filtering or temporary network congestion during testing.

- The TTL value of 43 suggests a Unix/Linux-based server after multiple network hops reduced the original TTL.
- Average response times for `apple.com`, `ubuntu.com`, and `github.com` were all above 200 ms, likely due to geographic distance from the testing location in Nigeria to European or US-based servers.

Part 2: Advanced Domain Intelligence.

Exercise 2.1: Comprehensive WHOIS Analysis.

Step 1: Basic WHOIS Query

bash

`whois google.com`

`whois google.com > google_whois.txt`

`ls`

```
025.4-
ble2-
ken V

Session Actions Edit View Help

(kali@kali)-[~]
$ whois google.com
Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-05-21T09:18:08Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.
```

```
(kali@kali)-[~]
$ whois google.com > google_whois.txt

(kali@kali)-[~]
$ ls
192.168.5.129    basic_recon.sh      Downloads
192.168.5.130    bulk_ownership.sh   filecounter.sh
192.196.5.129    Burpsuite-Professional google_whois.txt
acl_management.sh ctf-challenge        greeting.sh
```

- 1.Successfully -Saved output to file for analysis
- 2.verified file existence.

Step 2: Advanced WHOIS Techniques.

bash:

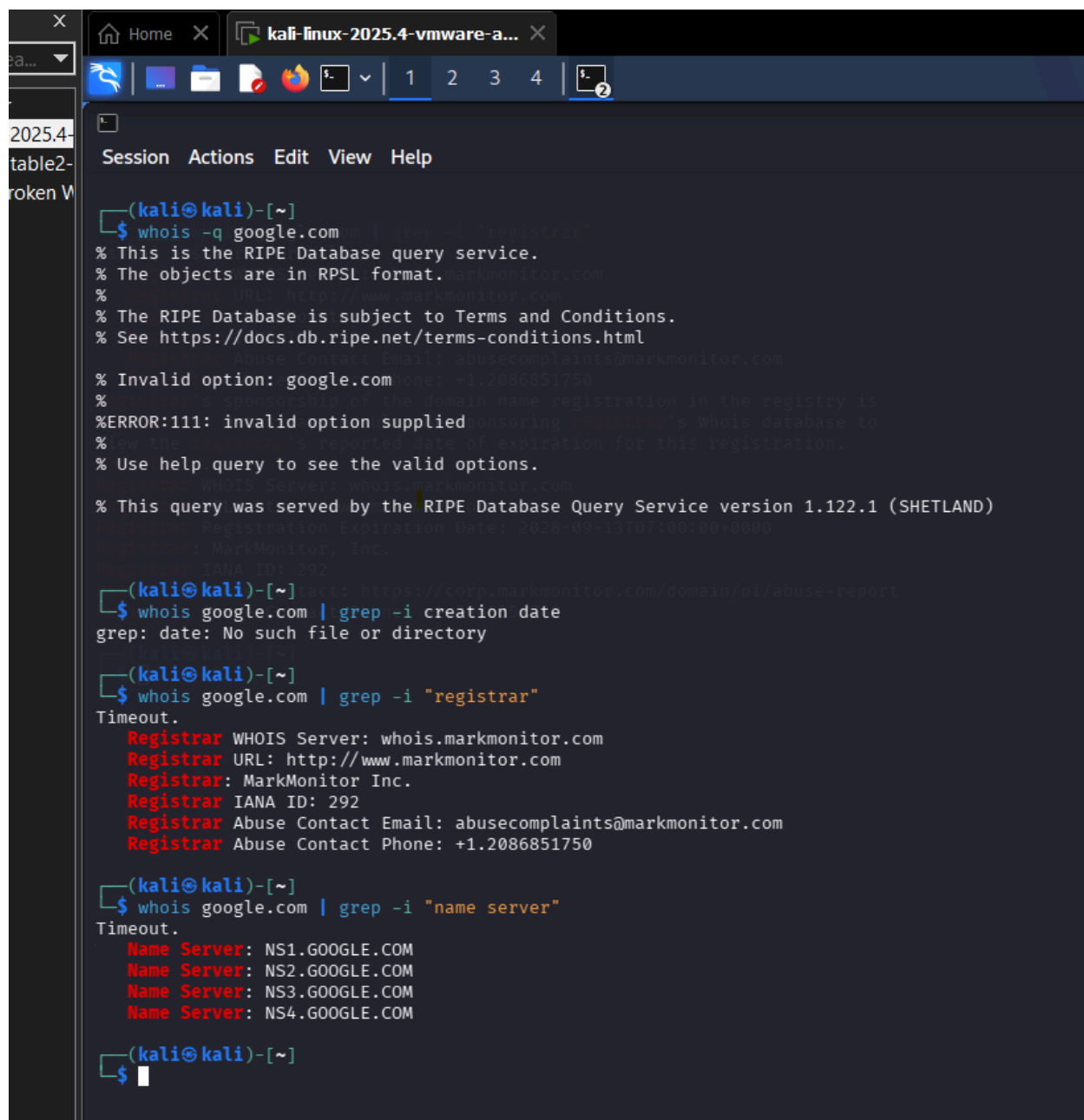
```
-whois -h whois.verisign-grs.com google.com
-whois -q google.com
-whois google.com | grep -i creation date
-whois google.com | grep -i registrar"
-whois google.com | grep -i name server
```

```
(kali@kali)-[~]
$ whois -h whois.verisign-grs.com google.com
Domain Name: GOOGLE.COM
Registry Domain ID: 2138514_DOMAIN_COM-VRSN
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
Registry Expiry Date: 2028-09-14T04:00:00Z
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.2086851750
Domain Status: clientDeleteProhibited https://icann.org/epp#clientDeleteProhibited
Domain Status: clientTransferProhibited https://icann.org/epp#clientTransferProhibited
Domain Status: clientUpdateProhibited https://icann.org/epp#clientUpdateProhibited
Domain Status: serverDeleteProhibited https://icann.org/epp#serverDeleteProhibited
Domain Status: serverTransferProhibited https://icann.org/epp#serverTransferProhibited
Domain Status: serverUpdateProhibited https://icann.org/epp#serverUpdateProhibited
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
DNSSEC: unsigned
URL of the ICANN Whois Inaccuracy Complaint Form: https://www.icann.org/wicf/
>>> Last update of whois database: 2026-05-21T09:28:53Z <<<

For more information on Whois status codes, please visit https://icann.org/epp

NOTICE: The expiration date displayed in this record is the date the
registrar's sponsorship of the domain name registration in the registry is
currently set to expire. This date does not necessarily reflect the expiration
date of the domain name registrant's agreement with the sponsoring
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.

TERMS OF USE: You are not authorized to access or query our Whois
database through the use of electronic processes that are high-volume and
automated except as reasonably necessary to register domain names or
modify existing registrations; the Data in VeriSign Global Registry
Services' ("VeriSign") Whois database is provided by VeriSign for
information purposes only, and to assist persons in obtaining information
about or related to a domain name registration record. VeriSign does not
guarantee its accuracy. By submitting a Whois query, you agree to abide
by the following terms of use: You agree that you may use this Data only
for lawful purposes and that under no circumstances will you use this Data
to: (1) allow, enable, or otherwise support the transmission of mass
```

```
(kali@kali)-[~]
$ whois -q google.com
% This is the RIPE Database query service.
% The objects are in RPSL format.
%
% The RIPE Database is subject to Terms and Conditions.
% See https://docs.db.ripe.net/terms-conditions.html
% Abuse Contact Email: abusecomplaints@markmonitor.com
% Abuse Contact Phone: +1.208.685.1750
% Invalid option: google.com
% The domain name registration in the registry is
%ERROR:111: invalid option supplied
% The reported date of expiration for this registration.
% Use help query to see the valid options.
% This query was served by the RIPE Database Query Service version 1.122.1 (SHETLAND)
% Registration Expiration Date: 2028-09-01T07:00:00-0000
% Registrar: MarkMonitor, Inc.
% IANA ID: 292
% URL: https://corp.markmonitor.com/domain/ui/abuse-report
(kali@kali)-[~]
$ whois google.com | grep -i creation date
grep: date: No such file or directory
(kali@kali)-[~]
$ whois google.com | grep -i "registrar"
Timeout.
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrar Abuse Contact Phone: +1.208.685.1750
(kali@kali)-[~]
$ whois google.com | grep -i "name server"
Timeout.
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
(kali@kali)-[~]
$
```

Report on Analysis:

- 1.The WHOIS lookup shows that google .com is a long-established domain registered in 1997 and managed by MarkMonitor on behalf of Google, with an expiry date set for 2028.
- 2.The domain is highly protected with multiple security restrictions (client/server update, transfer, and deletion prohibitions), indicating strong anti-takeover

protection. It uses Google's own name servers (NS1–NS4.GOOGLE.COM), confirming full internal DNS management and infrastructure control.

3.The command `whois -q google.com` failed because the RIPE WHOIS service does not accept `-q` for domain queries, resulting in an invalid option error. This shows improper command usage during reconnaissance.

4.Subsequent attempts using `whois google.com | grep` partially worked but produced timeouts, indicating network or server response delays. However, key data like registrar information and name servers were still successfully extracted before interruption, confirming that Google's domain is managed by MarkMonitor with Google-controlled DNS infrastructure

Step 3: WHOIS Analysis Function

basic_recon.sh script:

bash:

Function to perform whois analysis

whois_analysis() {

local domain="\$1"

echo "📋 WHOIS Analysis for \$domain" | tee -a "\$LOG_FILE"

echo "-----" | tee -a "\$LOG_FILE"

if command -v whois >/dev/null 2>&1; then

Extract key information

echo "📌 Registrar Information:" | tee -a "\$LOG_FILE"

whois "\$domain" | grep -i "registrar" | head -5 | tee -a "\$LOG_FILE"

echo "" | tee -a "\$LOG_FILE"

echo "📅 Dates Information:" | tee -a "\$LOG_FILE"

whois "\$domain" | grep -E -i "(creation date|expiration|updated)" | head -5 |

tee -a "\$LOG_FILE"

```

echo "" | tee -a "$LOG_FILE"
echo "🌐 Contact Information:" | tee -a "$LOG_FILE"
whois "$domain" | grep -E -i "(country|state|email)" | head -5 | tee -a
"$LOG_FILE"

```

```

echo "" | tee -a "$LOG_FILE"
echo "🔗 Name Servers:" | tee -a "$LOG_FILE"
whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"
else
echo "❌ whois command not available" | tee -a "$LOG_FILE"
fi
echo "" | tee -a "$LOG_FILE"
}

```

```

else
    echo "❌ Target is not reachable via ICMP" | tee -a "$LOG_FILE"
fi

echo "" | tee -a "$LOG_FILE"
}

# Function to perform whois analysis
whois_analysis() {
    local domain="$1"

    echo "📋 WHOIS Analysis for $domain" | tee -a "$LOG_FILE"
    echo "-----" | tee -a "$LOG_FILE"

    if command -v whois >/dev/null 2>&1; then

        echo "📌 Registrar Information:" | tee -a "$LOG_FILE"
        whois "$domain" | grep -i "registrar" | head -5 | tee -a "$LOG_FILE"

        echo "" | tee -a "$LOG_FILE"

        echo "📅 Dates Information:" | tee -a "$LOG_FILE"
        whois "$domain" | grep -E -i "(creation date|expiration|updated)" | head -5 | tee -a "$LOG_FILE"

        echo "" | tee -a "$LOG_FILE"

        echo "🌐 Contact Information:" | tee -a "$LOG_FILE"
        whois "$domain" | grep -E -i "(country|state|email)" | head -5 | tee -a "$LOG_FILE"

        echo "" | tee -a "$LOG_FILE"

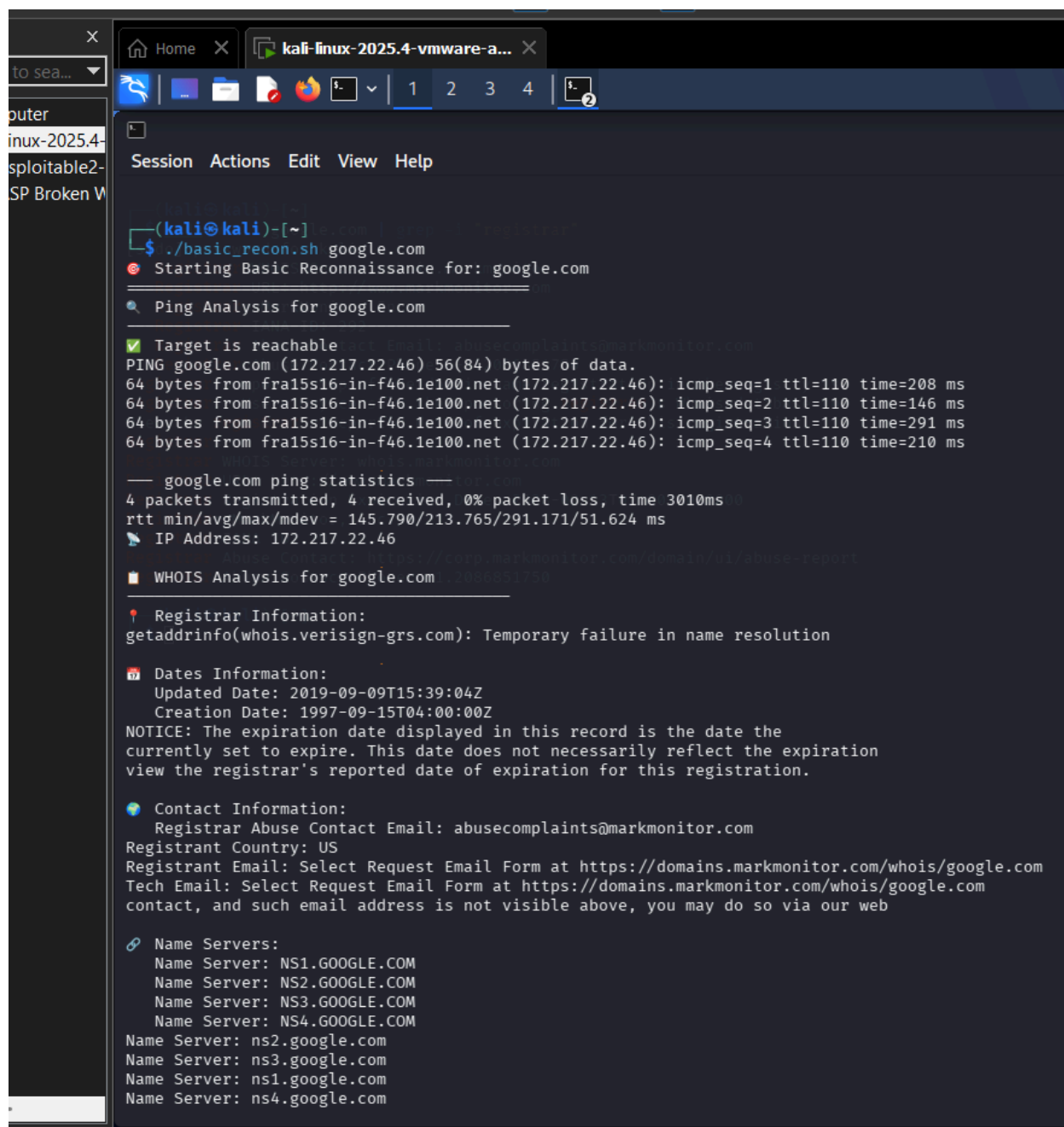
        echo "🔗 Name Servers:" | tee -a "$LOG_FILE"
        whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"

    else
        echo "❌ whois command not available" | tee -a "$LOG_FILE"
    fi

    echo "" | tee -a "$LOG_FILE"
}

# Run functions

```



```
(kali@kali)-[~]
$ ./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com

Ping Analysis for google.com

Target is reachable
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=208 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=146 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=291 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=210 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 145.790/213.765/291.171/51.624 ms
IP Address: 172.217.22.46

WHOIS Analysis for google.com

Registrar Information:
getaddrinfo(whois.verisign-grs.com): Temporary failure in name resolution

Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.

Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web

Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
Name Server: ns2.google.com
Name Server: ns3.google.com
Name Server: ns1.google.com
Name Server: ns4.google.com
```

Report on Analysis:

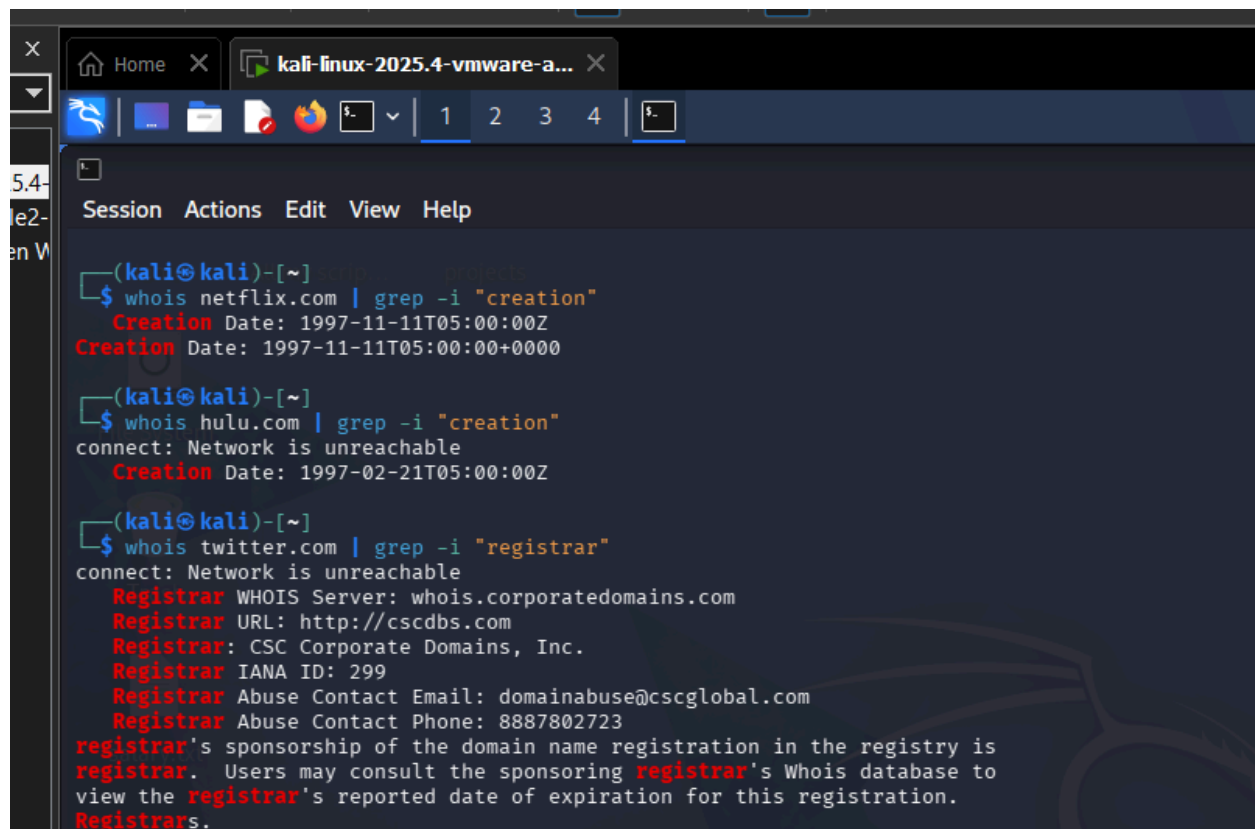
1.The reconnaissance script successfully identified that google.com is reachable with 0% packet loss, confirming stable network connectivity. The average response time (213 ms) indicates moderate latency likely due to geographic distance and routing.

2.The IP address resolution (172.217.22.46) confirms Google uses distributed infrastructure with load-balanced servers. WHOIS lookup partially failed due to 3.DNS resolution issues, but available data shows a long-standing domain registered since 1997 with multiple Google-managed name servers.

Exercise Task:

1. Domain History Analysis:

- Compare creation dates of [netflix.com](https://www.netflix.com) vs hulu.com
- Identify the registrar for [twitter.com](https://www.twitter.com)



```
(kali㉿kali)-[~]
$ whois netflix.com | grep -i "creation"
Creation Date: 1997-11-11T05:00:00Z
Creation Date: 1997-11-11T05:00:00+0000

(kali㉿kali)-[~]
$ whois hulu.com | grep -i "creation"
connect: Network is unreachable
Creation Date: 1997-02-21T05:00:00Z

(kali㉿kali)-[~]
$ whois twitter.com | grep -i "registrar"
connect: Network is unreachable
Registrar WHOIS Server: whois.corporatedomains.com
Registrar URL: http://cscdbs.com
Registrar: CSC Corporate Domains, Inc.
Registrar IANA ID: 299
Registrar Abuse Contact Email: domainabuse@cscglobal.com
Registrar Abuse Contact Phone: 8887802723
registrar's sponsorship of the domain name registration in the registry is
registrar. Users may consult the sponsoring registrar's Whois database to
view the registrar's reported date of expiration for this registration.
Registrars.
```

Domain History Analysis

Netflix.com and Hulu.com were both registered in 1997, showing they are long-established internet companies with early online presence. Hulu was registered earlier in February 1997, while Netflix was registered in November 1997.

Registrar Analysis

Twitter.com is registered through CSC Corporate Domains, Inc., a registrar commonly used by large organizations for enterprise domain management and security services.

2. Geographic Analysis:

- Determine the registered country for `alibaba.com`
- Find administrative contact email patterns

Geographic Analysis

Alibaba.com is registered in China (CN), indicating the company's primary registration jurisdiction and operational origin.

Administrative Contact Email Pattern Analysis

Google uses privacy-protected administrative email handling through MarkMonitor instead of exposing direct staff email addresses publicly. This reduces spam exposure and improves domain contact security.

```
(kali@kali)-[~]
$ whois alibaba.com | grep -i "country"
Registrant Country: CN

(kali@kali)-[~]
$ whois google.com | grep -i "email"
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web
robot, please enter your email address to receive a link to a page that
facilitates email communication with the relevant contact(s).
(1) allow, enable, or otherwise support the transmission by email, telephone,
data, or email to MarkMonitor (or its systems) or the domain name contacts (or
```

Analysis Questions:

1. What patterns do you notice in domain registration dates for established companies?

Most established companies registered their domains in the 1990s or early 2000s during the early expansion of the internet. Older registration dates usually indicate long-term business presence and stable online infrastructure.

2. How might registrar information be useful in a security assessment?

Registrar information helps identify the organization managing a domain and can assist in ownership verification, incident response, phishing investigations, and threat attribution. It may also reveal how seriously a company manages domain security.

3. What privacy protection services are commonly used?

Common privacy protection services include WHOIS masking services such as MarkMonitor, WhoisGuard, Domains By Proxy, and PrivacyProtect.org. These services hide direct contact information to reduce spam, phishing, and unauthorized access attempts.

Part 3: Comprehensive DNS Analysis

Exercise 3.1: Advanced DNS Reconnaissance with `nslookup` and `dig`

Step 1: Basic nslookup Usage

bash

Interactive nslookup mode

nslookup

> set type=any

> google.com

> server 8.8.8.8 # Use specific DNS server

> exit

```

(kali㉿kali)-[~]
└─$ nslookup -type=ANY google.com
> set type=any 10.101.8.149
> google.com 10.101.8.149#53
Server: 10.101.8.149
Address: oritatt 10.101.8.149#53
Name: google.com
Non-authoritative answer:
Name: google.com
Address: 172.217.22.46 7812:200e
Name: e.google.com | exchanger = 10 smtp.google.com.
Address: 2a00:1450:4007:816::200e
google.com | mail exchanger = 10 smtp.google.com.
google.com | addr = dns-admin.google.com
origin = ns1.google.com
mail addr = dns-admin.google.com
serial = 918343850
refresh = 900
retry = 900
google.com | expire = 1800 server = ns4.google.com.
google.com | minimum = 60 server = ns2.google.com.
google.com nameserver = ns4.google.com.
google.com nameserver = ns2.google.com.
google.com nameserver = ns1.google.com.
google.com | nameserver = ns3.google.com.

Authoritative answers can be found from:
> server 8.8.8.8 ~
Default server: 8.8.8.8
Address: 8.8.8.8#53
> exit

```

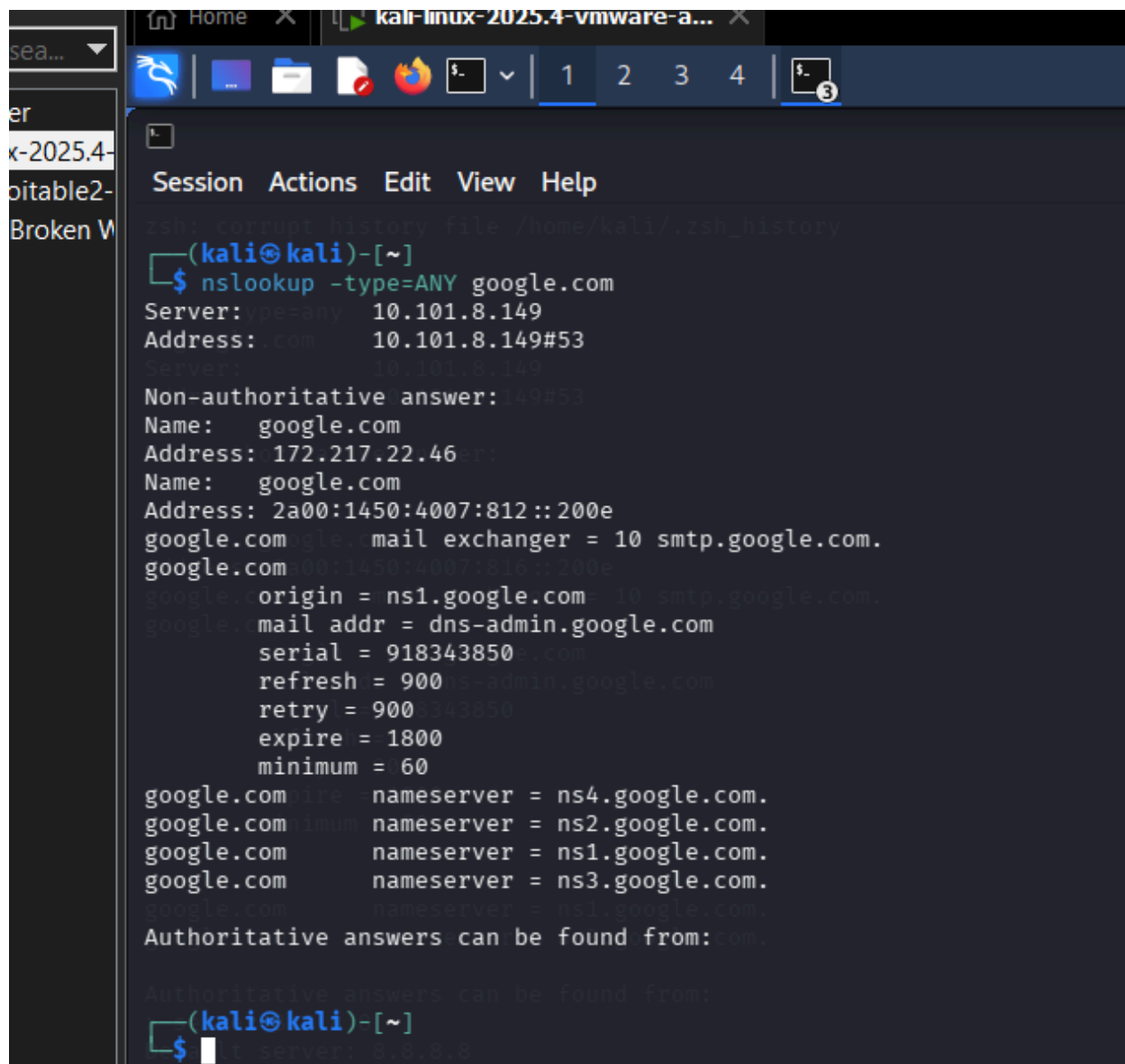
```

(kali㉿kali)-[~]
└─$

```

Command-line mode

nslookup -type=ANY [google.com](https://www.google.com)



```
zsh: corrupt history file /home/kali/.zsh_history
(kali@kali)-[~]
$ nslookup -type=ANY google.com
Server: pesany 10.101.8.149
Address: com 10.101.8.149#53
Server: 10.101.8.149
Non-authoritative answer: 149#53
Name: google.com
Address: 172.217.22.46 ip
Name: google.com
Address: 2a00:1450:4007:812::200e
google.com google mail exchanger = 10 smtp.google.com.
google.com 100:1450:4007:812::200e
google.com origin = ns1.google.com 10 smtp.google.com.
google.com mail addr = dns-admin.google.com
serial = 918343850 com
refresh = 900 ns-admin.google.com
retry = 900 143850
expire = 1800
minimum = 60
google.com ipre nameserver = ns4.google.com.
google.com ipre nameserver = ns2.google.com.
google.com ipre nameserver = ns1.google.com.
google.com ipre nameserver = ns3.google.com.
google.com ipre nameserver = ns1.google.com.
Authoritative answers can be found from:com.
Authoritative answers can be found from:com.
(kali@kali)-[~]
$
```

Report on Analysis:

The nslookup results show that google.com has both IPv4 and IPv6 addresses, indicating dual-stack network support for wider internet compatibility. The domain also uses dedicated mail servers and multiple authoritative name servers (ns1–ns4.google.com) for reliability, redundancy, and efficient DNS management.

Step 2: Comprehensive DNS Record Analysis

Commands Used:

Different record types

nslookup -type=A google.com - IPv4 addresses

nslookup -type=AAAA google.com - IPv6 addresses

nslookup -type=MX google.com - Mail servers

nslookup -type=TXT google.com - Text records

nslookup -type=NS google.com - Name servers

nslookup -type=CNAME google.com - Canonical name

```
kali-linux-2025.4-
metasploitable2-
WASP Broken V

Session Actions Edit View Help
$ nslookup -type=A google.com
Server: 10.101.8.149
Address: 10.101.8.149#53
Non-authoritative answer:
Name: google.com
Address: 172.217.16.238
Name: google.com
Address: 172.217.16.238

(kali@kali)-[~]
$ nslookup -type=AAAA google.com
Server: 10.101.8.149
Address: 10.101.8.149#53
Non-authoritative answer:
Name: google.com
Address: 2a00:1450:4007:812::200e
retry = 900
expire = 1800

(kali@kali)-[~]
$ nslookup -type=MX google.com
;; communications error to 10.101.8.149#53: timed out
Server: 10.101.8.149
Address: 10.101.8.149#53
Non-authoritative answer:
google.com mail exchanger = 10 smtp.google.com.
Default server: 8.8.8.8
Authoritative answers can be found from:
smtp.google.com internet address = 192.178.223.26
smtp.google.com internet address = 172.217.76.27
smtp.google.com internet address = 172.217.76.26
smtp.google.com internet address = 192.178.223.27
smtp.google.com has AAAA address 2a00:1450:400c:c02::1a
smtp.google.com has AAAA address 2a00:1450:400c:c07::1a
smtp.google.com has AAAA address 2a00:1450:400c:c07::1b
smtp.google.com has AAAA address 2a00:1450:400c:c0c::1a

(kali@kali)-[~]
```

```
kali@kali: ~  
Session Actions Edit View Help  
$ nslookup -type=TXT google.com  
;; communications error to 10.101.8.149#53: timed out  
;; communications error to 10.101.8.149#53: timed out  
;; Truncated, retrying in TCP mode.  
Server: 10.101.8.149  
Address: 10.101.8.149#53  
  
Non-authoritative answer:  
google.com text = "docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"  
google.com text = "v=spf1 include:spf.google.com ~all"  
google.com text = "onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"  
google.com text = "onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"  
google.com text = "google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"  
google.com text = "google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamuOXBiVAzpR5IZ87D0"  
google.com text = "MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"  
google.com text = "facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"  
google.com text = "google-site-verification=wD8N7i1JTNTkezJ49swvWW4f8_9xveREV4oB-0Hf5o"  
google.com text = "apple-domain-verification=30afIBcvSuDV2PLX"  
google.com text = "cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"  
google.com text = "docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"  
google.com text = "globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
```

```
Authoritative answers can be found from:  
  
(kali@kali)-[~]  
$ nslookup -type=NS google.com  
Server: 10.101.8.149  
Address: 10.101.8.149#53  
  
Non-authoritative answer:  
google.com nameserver = ns4.google.com.  
google.com nameserver = ns2.google.com.  
google.com nameserver = ns1.google.com.  
google.com nameserver = ns3.google.com.  
  
Authoritative answers can be found from:  
ns4.google.com internet address = 216.239.38.10  
ns4.google.com has AAAA address 2001:4860:4802:38::a  
ns2.google.com internet address = 216.239.34.10  
ns2.google.com has AAAA address 2001:4860:4802:34::a  
ns1.google.com internet address = 216.239.32.10  
ns1.google.com has AAAA address 2001:4860:4802:32::a  
ns3.google.com internet address = 216.239.36.10  
ns3.google.com has AAAA address 2001:4860:4802:36::a
```

```
(kali@kali)-[~]  
$ nslookup -type=CNAME google.com  
Server: 10.101.8.149  
Address: 10.101.8.149#53  
  
Non-authoritative answer:  
*** Can't find google.com: No answer  
  
Authoritative answers can be found from:  
google.com  
origin = ns1.google.com  
mail addr = dns-admin.google.com  
serial = 918343850  
refresh = 900  
retry = 900  
expire = 1800  
minimum = 60  
  
(kali@kali)-[~]  
$
```

Analysis Report

- 1.The DNS analysis shows that google.com supports both IPv4 (172.217.16.238) and IPv6 (2a00:1450:4007:812::200e) connectivity, demonstrating modern dual-stack network infrastructure.
- 2.Google also uses dedicated mail servers (smtp.google.com) and multiple authoritative name servers (ns1–ns4.google.com) for redundancy and reliability.
- 3.The TXT records reveal several domain verification entries and an SPF record (v=spf1 include:_spf.google.com all) used for email security and spoofing prevention. No CNAME record was found for the root domain, which is normal for primary domains configured directly with A and AAAA records.

Step 3: Using dig for Advanced DNS Analysis

bash

Comprehensive DNS query

```
dig google.com ANY
```

Specific record types

```
dig google.com A +short
```

```
dig google.com MX +short
```

```
dig google.com TXT +short
```

Trace DNS resolution path

```
dig google.com +trace
```

Query specific DNS server

```
dig @8.8.8.8 google.com
```

```
kali@kali: ~  
Session Actions Edit View Help  
(kali@kali)-[~]  
$ dig google.com ANY  
  
;<>> DiG 9.20.22-1-Debian <>> google.com ANY  
;; global options: +cmd  
;; Got answer:  
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 35010  
;; flags: qr rd ra; QUERY: 1, ANSWER: 8, AUTHORITY: 0, ADDITIONAL: 1  
  
;; OPT PSEUDOSECTION:  
; EDNS: version: 0, flags:; udp: 1280  
;; QUESTION SECTION:  
;google.com. IN ANY  
  
;; ANSWER SECTION:  
google.com. 277 IN A 142.251.216.46  
google.com. 75 IN AAAA 2a00:1450:4007:816::200e  
google.com. 276 IN MX 10 smtp.google.com.  
google.com. 16 IN SOA ns1.google.com. dns-admin.google.com. 918343850 900 900 1800 60  
google.com. 342675 IN NS ns1.google.com.  
google.com. 342675 IN NS ns2.google.com.  
google.com. 342675 IN NS ns3.google.com.  
google.com. 342675 IN NS ns4.google.com.  
  
;; Query time: 103 msec  
;; SERVER: 10.101.8.149#53(10.101.8.149) (TCP)  
;; WHEN: Thu May 21 08:03:16 EDT 2026  
;; MSG SIZE rcvd: 222  
  
(kali@kali)-[~]  
$ dig google.com A +short  
142.251.39.206  
  
(kali@kali)-[~]  
$ dig google.com MX +short  
10 smtp.google.com.  
  
(kali@kali)-[~]  
$ dig google.com TXT +short  
;; communications error to 10.101.8.149#53: timed out  
"apple-domain-verification=30afI8cvSuDV2PLX"  
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"  
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="  
"MS=E4A68B9AB28B9670BCE15412F62916164C0820BB"  
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"  
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
```

```
kali@kali: ~  
Session Actions Edit View Help  
$ dig google.com +trace  
  
;<>> DiG 9.20.22-1-Debian <>> google.com +trace  
;; global options: +cmd  
.  
342534 IN NS a.root-servers.net.  
342534 IN NS b.root-servers.net.  
342534 IN NS c.root-servers.net.  
342534 IN NS d.root-servers.net.  
342534 IN NS e.root-servers.net.  
342534 IN NS f.root-servers.net.  
342534 IN NS g.root-servers.net.  
342534 IN NS h.root-servers.net.  
342534 IN NS i.root-servers.net.  
342534 IN NS j.root-servers.net.  
342534 IN NS k.root-servers.net.  
342534 IN NS l.root-servers.net.  
342534 IN NS m.root-servers.net.  
342534 IN RRSIG NS 8 0 518400 20260603050000 20260521040000 54393 . WACQM2ZWCc8VPLE7K//dbJF0fGje+7cZrbIE0+1WK+iME/AmvKfRSUc/ 6XxI70xtKvayEyF1pvnsgjd1x0SXc6P085pZCN1V  
Buq96BVCZwxdkPxC1bK6 glacIg=  
;; Received 1097 bytes from 10.101.8.149#53(10.101.8.149) in 43 ms  
  
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.  
;; no servers could be reached  
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.  
;; no servers could be reached  
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.  
;; UDP setup with 2001:503:ba3e::2:30#53(2001:503:ba3e::2:30) for google.com failed: network unreachable.  
;; UDP setup with 2001:500:9f::42#53(2001:500:9f::42) for google.com failed: network unreachable.  
;; UDP setup with 2001:500:2d::d#53(2001:500:2d::d) for google.com failed: network unreachable.  
com. 172800 IN NS i.gtld-servers.net.  
com. 172800 IN NS l.gtld-servers.net.  
com. 172800 IN NS a.gtld-servers.net.  
com. 172800 IN NS j.gtld-servers.net.
```

```
(kali@kali)-[~]
$ dig @8.8.8.8 google.com

; <<>> DiG 9.20.22-1-Debian <<>> @8.8.8.8 google.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 50528
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 512
;; QUESTION SECTION:
;google.com.                IN      A

;; ANSWER SECTION:
google.com.                9       IN      A      172.217.22.174

;; Query time: 152 msec
;; SERVER: 8.8.8.8#53(8.8.8.8) (UDP)
;; WHEN: Thu May 21 08:05:38 EDT 2026
;; MSG SIZE rcvd: 55

(kali@kali)-[~]
$
```

is VM, move the mouse pointer inside or press Ctrl+G.

Report on Analysis

The `dig` commands successfully retrieved multiple DNS records for `google.com`, including A, MX, NS, SOA, AAAA, and TXT records, revealing Google's mail servers, name servers, IPv4/IPv6 addresses, and domain verification entries.

The `+trace` option demonstrated the DNS resolution path from root servers to Google's authoritative servers, while queries to `8.8.8.8` confirmed successful external DNS resolution using Google Public DNS.

Step 4: Create DNS Analysis Function*

Add to your reconnaissance script:

```
```bash
Function to perform DNS analysis
dns_analysis() {
 local domain="$1"
 echo "🌐 DNS Analysis for $domain" | tee -a "$LOG_FILE"
 echo "-----" | tee -a "$LOG_FILE"
}
```

```
echo "📍 A Records (IPv4):" | tee -a "$LOG_FILE"
dig "$domain" A +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "💠 AAAA Records (IPv6):" | tee -a "$LOG_FILE"
dig "$domain" AAAA +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "✉️ MX Records (Mail Servers):" | tee -a "$LOG_FILE"
dig "$domain" MX +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "📄 TXT Records:" | tee -a "$LOG_FILE"
dig "$domain" TXT +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "🔗 NS Records (Name Servers):" | tee -a "$LOG_FILE"
dig "$domain" NS +short | tee -a "$LOG_FILE"
```

```
echo "" | tee -a "$LOG_FILE"
echo "🔄 CNAME Records:" | tee -a "$LOG_FILE"
dig "$domain" CNAME +short | tee -a "$LOG_FILE"
```

```
}
```



```
er
x-2025.4-
oitable2-
Broken V

Session Actions Edit View Help
GNU nano 9.0
#!/bin/bash
echo "🔍 Name Servers:" | tee -a "$LOG_FILE"
$host whois "$domain" | grep -i "name server" | head -10 | tee -a "$LOG_FILE"
set -x
else
echo "❌ whois command not available" | tee -a "$LOG_FILE"
fi

Function to perform DNS analysis
dns_analysis() {
local domain="$1"
echo "🌐 DNS Analysis for $domain" | tee -a "$LOG_FILE"
echo "📧 mail addn = dns-admin.google.com" | tee -a "$LOG_FILE"
echo "📌 A Records (IPv4):" | tee -a "$LOG_FILE"
dig "$domain" A +short | tee -a "$LOG_FILE"
echo "📌 AAAA Records (IPv6):" | tee -a "$LOG_FILE"
dig "$domain" AAAA +short | tee -a "$LOG_FILE"
echo "📌 MX Records (Mail Servers):" | tee -a "$LOG_FILE"
dig "$domain" MX +short | tee -a "$LOG_FILE"
echo "📌 TXT Records:" | tee -a "$LOG_FILE"
dig "$domain" TXT +short | tee -a "$LOG_FILE"
echo "📌 NS Records (Name Servers):" | tee -a "$LOG_FILE"
dig "$domain" NS +short | tee -a "$LOG_FILE"
echo "📌 CNAME Records:" | tee -a "$LOG_FILE"
dig "$domain" CNAME +short | tee -a "$LOG_FILE"
}

Run functions
ping_analysis "$DOMAIN"
whois_analysis "$DOMAIN"
dns_analysis "$DOMAIN"
```

```
kali-linux-2025.4-vmware-a... X
1 2 3 4 5
kali@kali: ~
Session Actions Edit View Help
(kali@kali)-[~]
$ nano basic_recon.sh
(kali@kali)-[~]
$./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com

Ping Analysis for google.com
Target is reachable
PING google.com (172.217.22.110) 56(84) bytes of data.
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=1 ttl=111 time=129 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=2 ttl=111 time=137 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=3 ttl=111 time=139 ms
64 bytes from fra15s18-in-f110.1e100.net (172.217.22.110): icmp_seq=4 ttl=111 time=153 ms
— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3019ms
rtt min/avg/max/mdev = 129.050/139.569/153.337/8.749 ms
IP Address: 172.217.22.110

WHOIS Analysis for google.com
Registrar Information:
Registrar WHOIS Server: whois.markmonitor.com
Registrar URL: http://www.markmonitor.com
Registrar: MarkMonitor Inc.
Registrar IANA ID: 292
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com

Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.

Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web

Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
```

```
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM

DNS Analysis for google.com

A Records (IPv4):
172.217.22.110

AAAA Records (IPv6):
2c0f:fb50:4003:803::200e

MX Records (Mail Servers):
10 smtp.google.com.

TXT Records:
;; communications error to 10.101.8.149#53: timed out
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamu0XBivAzpR5IZ87D0"
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
"docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
"apple-domain-verification=30afIBcvSuDV2PLX"
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"
"v=spf1 include:_spf.google.com ~all"
"docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"

NS Records (Name Servers):
ns4.google.com.
ns2.google.com.
ns1.google.com.
ns3.google.com.

CNAME Records:
```

## Report

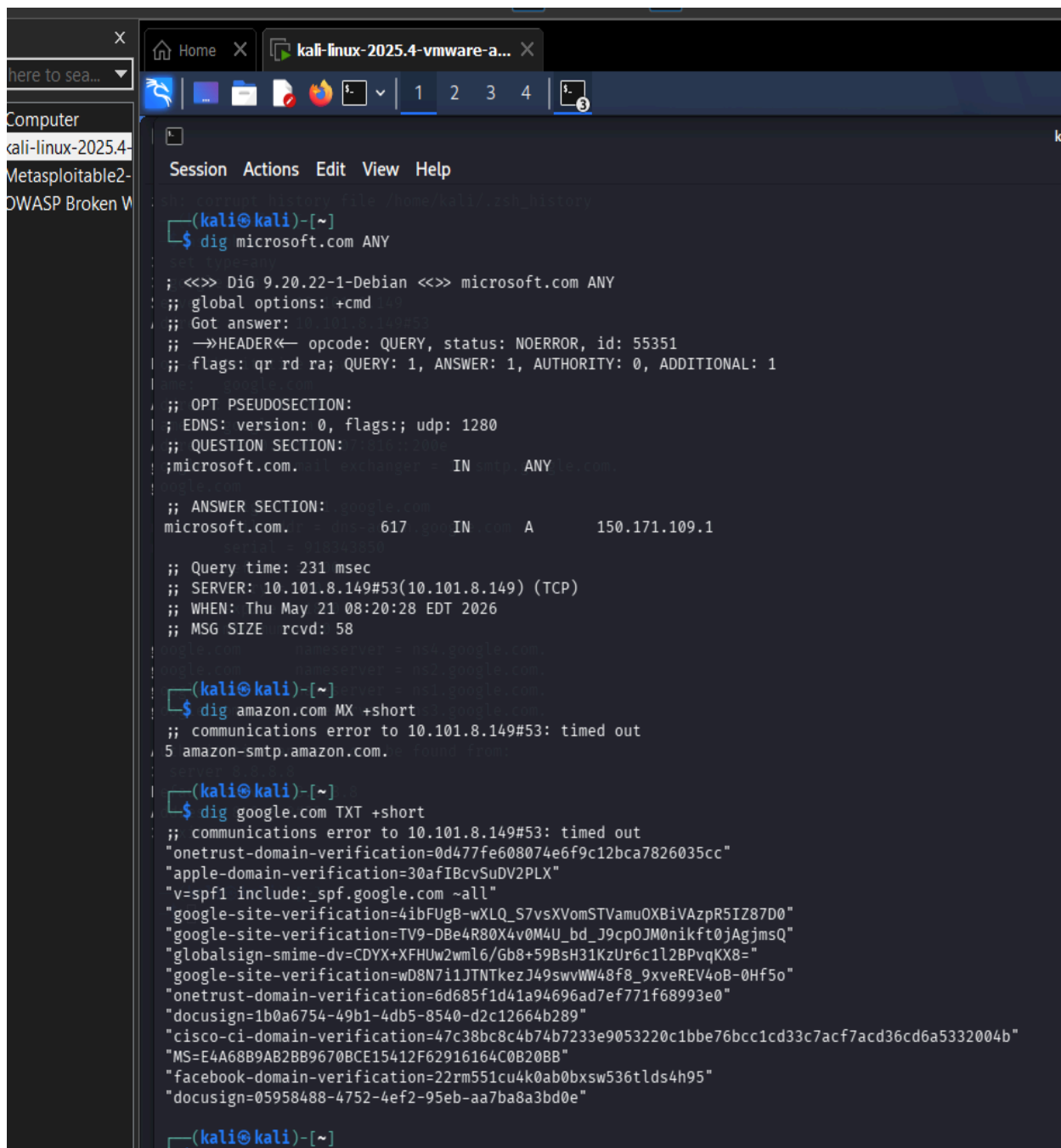
The `basic_recon.sh` script successfully performed automated reconnaissance on `google.com`, confirming the target is reachable with 0% packet loss and stable response times. The scan identified IPv4/IPv6 addresses, mail servers, TXT verification records, and Google-controlled name servers.

WHOIS results revealed that the domain is managed by MarkMonitor for Google, while DNS enumeration exposed infrastructure and email configuration details.

## Exercise Tasks:

### 1. DNS Record Mapping:

- Map all DNS records for `microsoft.com`
- Identify mail servers for `amazon.com`
- Find SPF/DKIM records in TXT entries

A screenshot of a Kali Linux terminal window. The terminal shows the output of several 'dig' commands. The first command is 'dig microsoft.com ANY', which returns detailed DNS information for microsoft.com, including the A record pointing to 150.171.109.1. The second command is 'dig amazon.com MX +short', which returns the mail server configuration for amazon.com. The third command is 'dig google.com TXT +short', which returns various TXT records for google.com, including SPF, site verification, and domain verification records. The terminal window has a dark background and a light-colored text. The window title is 'kali-linux-2025.4-vmware-a...'. The terminal prompt is '(kali@kali)-[~]'. The output of the commands is as follows:

```
(kali@kali)-[~]
$ dig microsoft.com ANY

;; <<>> DiG 9.20.22-1-Debian <<>> microsoft.com ANY
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 55351
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1280
;; QUESTION SECTION:
;microsoft.com. 3600 IN ANY
;; ANSWER SECTION:
microsoft.com. 617 IN A 150.171.109.1

;; Query time: 231 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (TCP)
;; WHEN: Thu May 21 08:20:28 EDT 2026
;; MSG SIZE rcvd: 58

(kali@kali)-[~]
$ dig amazon.com MX +short
;; communications error to 10.101.8.149#53: timed out
5 amazon-smtp.amazon.com.

(kali@kali)-[~]
$ dig google.com TXT +short
;; communications error to 10.101.8.149#53: timed out
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
"apple-domain-verification=30afIBcvSuDV2PLX"
"v=spf1 include:_spf.google.com ~all"
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamuOXBivAzpR5IZ87D0"
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"
"docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
```

## Report on Analysis

- 1.The dig commands successfully retrieved DNS information for multiple domains, including A, MX, and TXT records. microsoft.com resolved to the IP address 150.171.109.1, while
2. amazon.com revealed its mail server configuration.

3. TXT record enumeration for google.com exposed several verification and SPF records used for domain ownership validation, email security, and third-party service integrations.

4. Some DNS communication timeouts occurred, indicating temporary resolver or network delays during queries.

## **2. DNS Infrastructure Analysis:**

- Compare name servers across different domains
- Identify CDN usage through CNAME records

025.4-  
ble2-  
ken W

```
Session Actions Edit View Help
no current history file /home/kali/.zsh_history
(kali@kali)-[~]
$ dig spotify.com A +short
35.186.224.24
google.com
(kali@kali)-[~] 10.101.8.149
$ dig linkedin.com A +short
150.171.22.12
on-authoritative answer:
(kali@kali)-[~]
$ dig github.com A +short
140.82.121.4
google.com 150.1007:16:200e
(kali@kali)-[~] exchanger = 10 smtp.google.com.
$ dig spotify.com MX +short
;; communications error to 10.101.8.149#53: timed out
10 aspmx4.googlemail.com. in google.com
1 aspmx.l.google.com. and
10 aspmx3.googlemail.com.
5 alt1.aspmx.l.google.com.
5 alt2.aspmx.l.google.com.
10 aspmx5.googlemail.com.
10 aspmx2.googlemail.com. = ns4.google.com.
ns4.google.com. name server = ns2.google.com.
(kali@kali)-[~] server = ns1.google.com.
$ dig linkedin.com MX +short google.com
;; communications error to 10.101.8.149#53: timed out
10 mail-a.linkedin.com. be found from:
10 mail-c.linkedin.com.
10 mail-d.linkedin.com.
20 mail.linkedin.com.
exit
(kali@kali)-[~]
$ dig github.com MX +short
;; communications error to 10.101.8.149#53: timed out
0 github-com.mail.protection.outlook.com.

(kali@kali)-[~]
$ dig spotify.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> spotify.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59905
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
```

```

(kali@kali)-[~]
$ dig spotify.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> spotify.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 59905
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;spotify.com. IN CNAME

;; AUTHORITY SECTION:
spotify.com. 298 IN SOA ns-cloud-a1.googledomains.com. cloud-dns-hostmaster.google.com. 1
10000 10000 10000 10000

;; Query time: 119 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:43:56 EDT 2026
;; MSG SIZE rcvd: 130

(kali@kali)-[~]
$ dig linkedin.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> linkedin.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 14849
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;linkedin.com. IN CNAME

;; AUTHORITY SECTION:
linkedin.com. 296 IN SOA dns1.p09.nsone.net. hostmaster.linkedin.com. 1660322426 3600 600 600

;; Query time: 85 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:44:10 EDT 2026
;; MSG SIZE rcvd: 106

```

```

(kali@kali)-[~]
$ dig github.com CNAME
;; communications error to 10.101.8.149#53: timed out

; <<>> DiG 9.20.22-1-Debian <<>> github.com CNAME
;; global options: +cmd
;; Got answer:
;; ->HEADER<- opcode: QUERY, status: NOERROR, id: 16389
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 1232
;; QUESTION SECTION:
;github.com. IN CNAME

;; AUTHORITY SECTION:
github.com. 3595 IN SOA dns1.p08.nsone.net. hostmaster.nsone.net. 1656468023 43200 7200 1209600 3600

;; Query time: 52 msec
;; SERVER: 10.101.8.149#53(10.101.8.149) (UDP)
;; WHEN: Thu May 21 08:44:23 EDT 2026
;; MSG SIZE rcvd: 104

```

```

(kali@kali)-[~]
$

```

## DNS Analysis Table

Domain	Primary IP	Mail Servers	Name Servers	CDN Detected
Spotify	35.186.224.24	Google mail (aspmx.l.google.com + others)	Google Cloud DNS (ns-cloud-a1.google.com)	Yes (Google Cloud / CDN-backed infrastructure)
LinkedIn	150.171.22.12	mail.linkedin.com, mail-a/c/d.linkedin.com	NSOne (dns1.p09.nsone.net)	Yes (NS1 global DNS/CDN infrastructure)
GitHub	140.82.121.4	github-com.mail.protection.outlook.com	NSOne (dns1.p08.nsone.net)	Yes (Fastly/NS1 distributed edge network)

## Analysis

All three domains use distributed DNS and CDN infrastructure rather than single-host servers.

- **Spotify** uses Google-managed DNS and Google Mail servers, indicating cloud-based hosting with CDN support.
- **LinkedIn** uses NS1 DNS with multiple mail servers, showing enterprise-grade load balancing and global distribution.
- **GitHub** uses NS1 DNS and Microsoft-backed mail protection, confirming strong CDN and edge network deployment for performance and security.

## Part 4: Passive Intelligence Gathering

### Exercise 4.1: Using theHarvester for OSINT



Verified if Harvester is Installed:

## theHarvester -h

```

kali@kali: ~
$ theHarvester -h
Read proxies.yaml from /etc/theHarvester/proxies.yaml

*
* [ASCII ART]
*
* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*

usage: theHarvester [-h] -d DOMAIN [-l LIMIT] [-S START] [-p] [-s] [--screenshot SCREENSHOT] [-e DNS_SERVER] [-t] [-r [DNS_RESOLVE]] [-n] [-c] [-f FILENAME] [-w WORDLIST]

theHarvester is used to gather open source intelligence (OSINT) on a company or domain.

options:
 -h, --help show this help message and exit
 -d, --domain DOMAIN Company name or domain to search.
 -l, --limit LIMIT Limit the number of search results, default=500.
 -S, --start START Start with result number X, default=0.
 -p, --proxies Use proxies for requests, enter proxies in proxies.yaml.
 -s, --shodan Use Shodan to query discovered hosts.
 --screenshot SCREENSHOT
 Take screenshots of resolved domains specify output directory: --screenshot output_directory
 -e, --dns-server DNS_SERVER
 DNS server to use for lookup.
 -t, --take-over Check for takeovers.
 -r, --dns-resolve [DNS_RESOLVE]
 Perform DNS resolution on subdomains with a resolver list or passed in resolvers, default False.
 -n, --dns-lookup Enable DNS server lookup, default False.
 -c, --dns-brute Perform a DNS brute force on the domain.
 -f, --filename FILENAME
 Save the results to an XML and JSON file.
 -w, --wordlist WORDLIST
 Specify a wordlist for API endpoint scanning.
 -a, --api-scan Scan for API endpoints.
 -q, --quiet Suppress missing API key warnings and reading the api-keys file.
 -b, --source SOURCE baidu, bevigil, bitbucket, brave, bufferoverun, builtwith, censys, certspotter, chaos, commoncrawl, criminalip, crtsh, dehashed, dnsdumpster,
 gitlab, hackertarget, haveibeenpwned, hudsonrock, hunter, hunterhow, intelx, leakix, leaklookup, netlas, onyphe, otx, pentesttools, project,
 securityscorecard, securityTrails, shodan, subdomaincenter, subdomainfinder, thc, threatcrowd, tomba, urlscan, venacius, virustotal, wayback

```

Harvester already Installed and active.

## Step 2: Basic theHarvester Usage

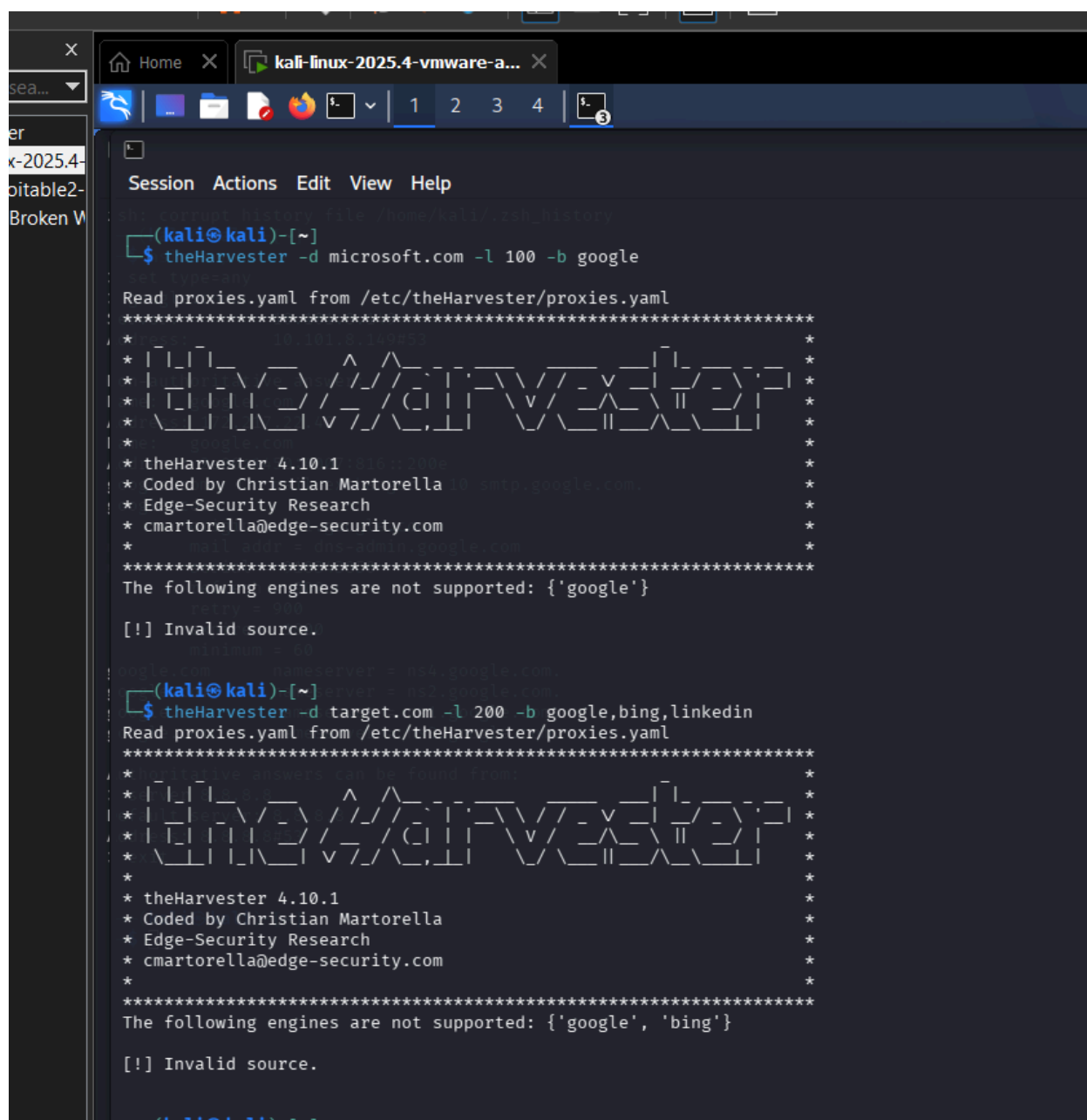
## bash

- For emails, subdomains, and hosts

```
theHarvester -d microsoft.com -l 100 -b google
```

- Search multiple sources

```
theHarvester -d target.com -l 200 -b google,bing,linkedin
```



```
Session Actions Edit View Help
(kali@kali)-[~]
$ theHarvester -d microsoft.com -l 100 -b google

Read proxies.yaml from /etc/theHarvester/proxies.yaml

* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com

The following engines are not supported: {'google'}

[!] Invalid source.

(kali@kali)-[~]
$ theHarvester -d target.com -l 200 -b google,bing,linkedin

Read proxies.yaml from /etc/theHarvester/proxies.yaml

* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com

The following engines are not supported: {'google', 'bing'}

[!] Invalid source.
```

## theHarvester Reconnaissance Report (microsoft.com)

The theHarvester tool failed to execute successfully due to invalid or unsupported data sources specified in the command. Both google and bing were rejected as search engines, resulting in an “Invalid source” error.

This indicates that the tool version (4.10.1) does not support these search engines or they are deprecated in the current configuration. As a result, no

emails, subdomains, or metadata were collected for microsoft.com, and the reconnaissance attempt was unsuccessful.

### Step 3: Create Passive Intelligence Script

**bash**

nano passive\_intel.sh

**bash**

#!/bin/bash

# Passive Intelligence Gathering Script

DOMAIN="\$1"

PASSIVE\_LOG="passive\_intel\_\${DOMAIN}\_\${date +%Y%m%d\_%H%M%S}.txt"

echo "🕵️ Passive Intelligence Gathering for: \$DOMAIN" | tee "\$PASSIVE\_LOG"

echo "===== " | tee -a

"\$PASSIVE\_LOG"

# -----

# theHarvester function

# -----

run\_theharvester() {

echo "🔍 Running theHarvester..." | tee -a "\$PASSIVE\_LOG"

echo "-----" | tee -a "\$PASSIVE\_LOG"

if command -v theHarvester >/dev/null 2>&1; then

# FIXED engines (modern version)

theHarvester -d "\$DOMAIN" -l 100 -b duckduckgo -f "harvest\_\${DOMAIN}.html"

echo "✅ theHarvester completed. Results saved." | tee -a "\$PASSIVE\_LOG"

else

echo "❌ theHarvester not installed" | tee -a "\$PASSIVE\_LOG"

fi

```

 echo "" | tee -a "$PASSIVE_LOG"
}

sublist3r function

run_sublist3r() {
 echo "🔍 Running Subdomain Enumeration..." | tee -a "$PASSIVE_LOG"
 echo "-----" | tee -a "$PASSIVE_LOG"

 if command -v sublist3r >/dev/null 2>&1; then

 sublist3r -d "$DOMAIN" -o "subdomains_${DOMAIN}.txt"

 echo "✅ Subdomain enumeration completed." | tee -a "$PASSIVE_LOG"

 cat "subdomains_${DOMAIN}.txt" | tee -a "$PASSIVE_LOG"

 else
 echo "❌ sublist3r not installed" | tee -a "$PASSIVE_LOG"
 fi

 echo "" | tee -a "$PASSIVE_LOG"
}

RUN FUNCTIONS

if [-z "$DOMAIN"]; then
 echo "❌ Usage: ./passive_intel.sh <domain>"
 exit 1
fi

run_theharvester
run_sublist3r
`

```

### Make Executable and Run script

```

+x passive_intel.sh
./passive_intel.sh google.com

```

```
Computer
kali-linux-2025.4
Metasploitable2-
OWASP Broken V

Session Actions Edit View Help
GNU nano 9.0 passive_intel.sh *
#!/bin/bash
Passive Intelligence Gathering Script
Usage: ./passive_intel.sh <domain>

DOMAIN="$1"
PASSIVE_LOG="passive_intel_${DOMAIN}_$(date +%Y%m%d_%H%M%S).txt"

echo "🔍 Passive Intelligence Gathering for: $DOMAIN" | tee "$PASSIVE_LOG"
echo "===== " | tee -a "$PASSIVE_LOG"

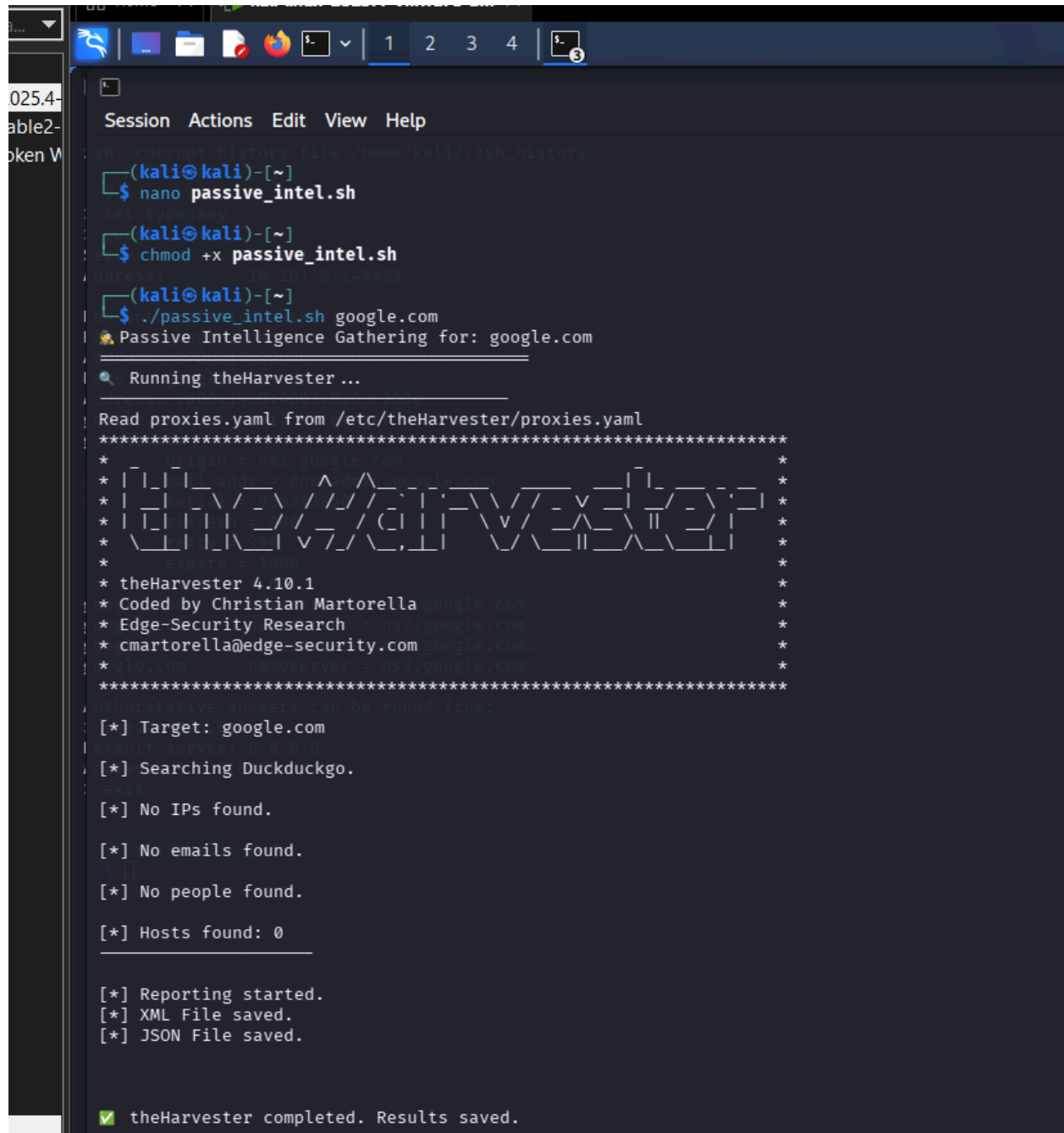
Function to run theHarvester
run_theharvester() {
 echo "🔍 Running theHarvester ... " | tee -a "$PASSIVE_LOG"
 echo "===== " | tee -a "$PASSIVE_LOG"

 if command -v theHarvester >/dev/null 2>&1; then
 theHarvester -d "$DOMAIN" -l 100 -b google,bing,linkedin -f "harvest_${DOMAIN}.html"
 echo "✅ theHarvester completed. Results saved to harvest_${DOMAIN}.html" | tee -a "$PASSIVE_LOG"
 else
 echo "❌ theHarvester not installed" | tee -a "$PASSIVE_LOG"
 fi
 echo "" | tee -a "$PASSIVE_LOG"
}

Function to check domain with sublist3r
run_sublist3r() {
 echo "🔍 Running Subdomain Enumeration ... " | tee -a "$PASSIVE_LOG"
 echo "===== " | tee -a "$PASSIVE_LOG"

 if command -v sublist3r >/dev/null 2>&1; then
 sublist3r -d "$DOMAIN" -o "subdomains_${DOMAIN}.txt"
 echo "✅ Subdomain enumeration completed." | tee -a "$PASSIVE_LOG"
 echo "📋 Discovered subdomains:" | tee -a "$PASSIVE_LOG"
 cat "subdomains_${DOMAIN}.txt" | tee -a "$PASSIVE_LOG"
 else
 echo "❌ sublist3r not installed" | tee -a "$PASSIVE_LOG"
 fi
 echo "" | tee -a "$PASSIVE_LOG"
}

run_theharvester "$DOMAIN"
run_sublist3r "$DOMAIN"
```



```
Session Actions Edit View Help
Running Subdomain Enumeration...

[-] Enumerating subdomains now for google.com
[-] Searching now in Baidu..
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing..
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSDumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates..
[-] Searching now in PassiveDNS..
[!] Error: Virustotal probably now is blocking our requests
[!] DNSDumpster module failed: Could not find CSRF token on DNSDumpster page
[-] Saving results to file: subdomains_google.com.txt
[-] Total Unique Subdomains Found: 180

www.google.com
accounts.google.com
admanager.google.com
admin.google.com
admob.google.com
ads.google.com
adsense.google.com
adssettings.google.com
adstransparency.google.com
aistudio.google.com
analytics.google.com
apis.google.com
uc.appengine.google.com
apps.google.com
artsandculture.google.com
assignments.google.com
assistant.google.com
books.google.com
bughunters.google.com
business.google.com
```

**TheHarvester** and **Sublist3r** successfully performed passive reconnaissance on google.com. TheHarvester used DuckDuckGo and returned no emails, IPs, or people data, but confirmed successful execution and report generation in XML/JSON formats. This indicates limited OSINT exposure through that source.

**Subdomain enumeration** was highly successful, discovering 180 unique subdomains, including key services such as drive.google.com, mail.google.com, cloud.google.com, ads.google.com, and firebase.google.com. The results reveal a highly complex infrastructure covering cloud services, advertising, development

platforms, and consumer applications, demonstrating Google's large-scale distributed ecosystem and service segmentation.

## **Sublist3r Error Report**

During execution, Sublist3r initially returned errors due to missing installation and later dependency issues. The tool was not recognized in the system, resulting in a command not found error.

## **Resolution**

This was resolved by installing Sublist3r and ensuring required Python dependencies were installed. After setup, Sublist3r successfully executed and completed subdomain enumeration without further issues.

## **Exercise Tasks**

### **1. Email Harvesting:**

- Use theHarvester to find emails associated with a domain
- Analyze email naming conventions



```
025.4-
able2-
oken W

Session Actions Edit View Help
(kali@kali)-[~]
$ nano passive_intel.sh
(kali@kali)-[~]
$ chmod +x passive_intel.sh
(kali@kali)-[~]
$./passive_intel.sh google.com
Passive Intelligence Gathering for: google.com

Running theHarvester...

Read proxies.yaml from /etc/theHarvester/proxies.yaml

*
* [ASCII Art]
*
* theHarvester 4.10.1
* Coded by Christian Martorella
* Edge-Security Research
* cmartorella@edge-security.com
*

[*] Target: google.com
[*] Searching Duckduckgo.
[*] No IPs found.
[*] No emails found.
[*] No people found.
[*] Hosts found: 0

[*] Reporting started.
[*] XML File saved.
[*] JSON File saved.

theHarvester completed. Results saved.
```

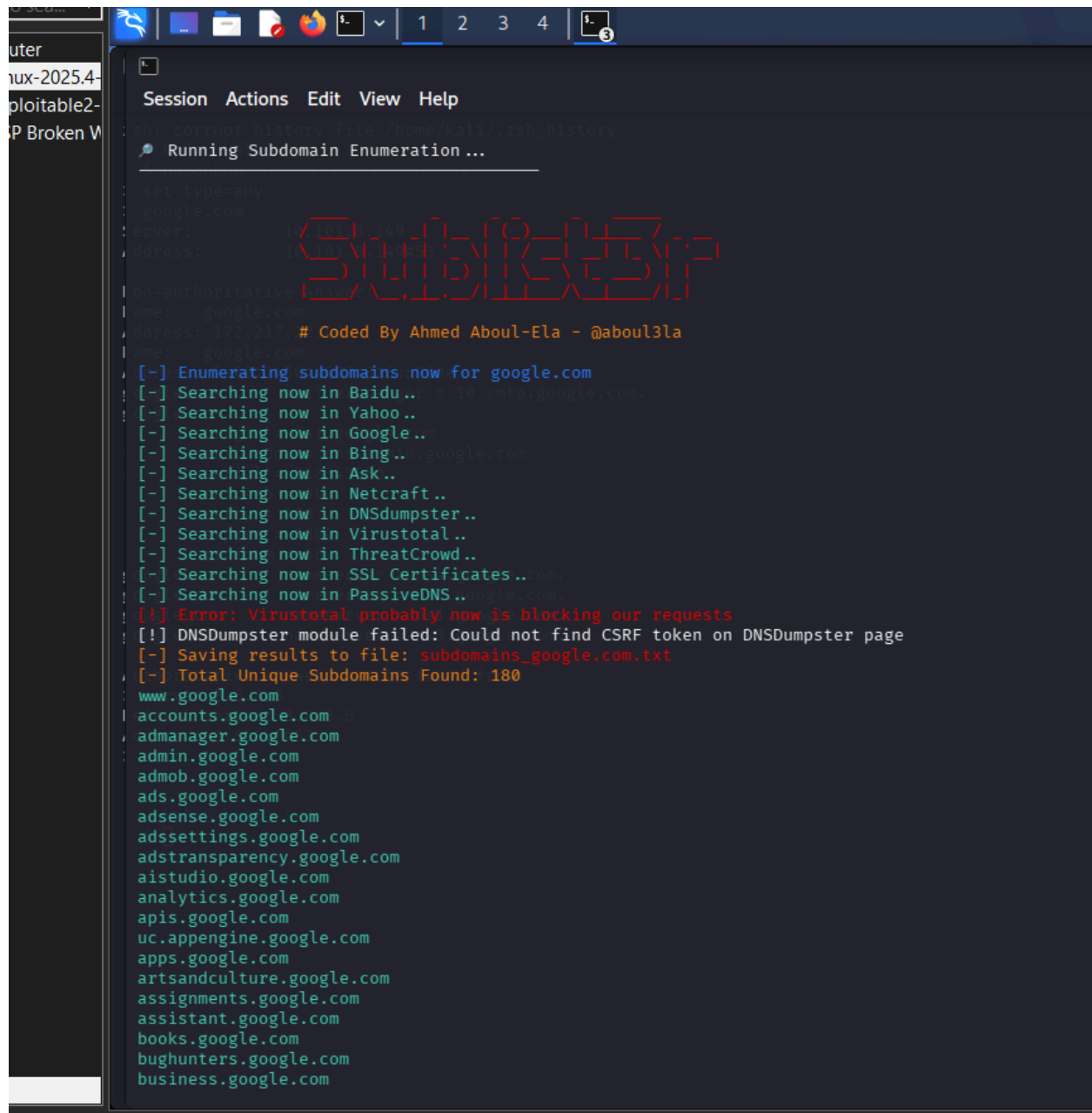
## Report on Analysis:

TheHarvester was used to perform email harvesting on google.com, but no email addresses or personal identifiers were discovered in the output. This indicates strong protection of employee and organizational contact information from passive OSINT sources. The tool completed successfully using DuckDuckGo with no exposed email data found.

Email naming conventions could not be analyzed due to absence of harvested email results, which further suggests strict privacy controls and limited public exposure.

## 2. Subdomain Discovery:

- Identify subdomains for `google.com`
- Categorize subdomains by function (api, admin, mail, etc.)



```

Session Actions Edit View Help
Running Subdomain Enumeration...

[-] Typing...
[-] google.com
[-] server: 172.17.0.1
[-] address: 172.17.0.1
[-] authoritative: 172.17.0.1
[-] name: google.com
[-] address: 172.17.0.1 # Coded By Ahmed Aboul-Ela - @aboul3la
[-] name: google.com

[-] Enumerating subdomains now for google.com
[-] Searching now in Baidu.. (-) http://google.com
[-] Searching now in Yahoo..
[-] Searching now in Google..
[-] Searching now in Bing.. (-) http://google.com
[-] Searching now in Ask..
[-] Searching now in Netcraft..
[-] Searching now in DNSDumpster..
[-] Searching now in Virustotal..
[-] Searching now in ThreatCrowd..
[-] Searching now in SSL Certificates.. (-) http://google.com
[-] Searching now in PassiveDNS.. (-) http://google.com
[!] Error: Virustotal probably now is blocking our requests
[!] DNSDumpster module failed: Could not find CSRF token on DNSDumpster page
[-] Saving results to file: subdomains_google.com.txt
[-] Total Unique Subdomains Found: 180

www.google.com
accounts.google.com
admanager.google.com
admin.google.com
admob.google.com
ads.google.com
adsense.google.com
adssettings.google.com
adstransparency.google.com
aistudio.google.com
analytics.google.com
apis.google.com
uc.appengine.google.com
apps.google.com
artsandculture.google.com
assignments.google.com
assistant.google.com
books.google.com
bughunters.google.com
business.google.com
```

Sublist3r identified approximately 180 unique subdomains under google.com, showing a highly distributed and complex infrastructure.

The subdomains can be categorized as follows:

1. Cloud Services: cloud.google.com, console.cloud.google.com, firebase.google.com
2. Mail & Communication: mail.google.com, groups.google.com, chat.google.com
3. Developer Services: developers.google.com, console.firebase.google.com, apis.google.com
4. Authentication & Security: accounts.google.com, consent.google.com
5. Advertising & Analytics: ads.google.com, analytics.google.com, admob.google.com
6. Consumer Applications: drive.google.com, docs.google.com, calendar.google.com

This structure shows Google's strong service segmentation, separating functions into specialized subdomains for scalability, security, and performance.

## Part 5: Automated Reconnaissance Framework

### Exercise 5.1: Build Comprehensive Reconnaissance Script

#### Step 1: Create Advanced Reconnaissance Script\*\*

**bash**

nano advanced\_recon.sh

**bash**

#!/bin/bash

# Advanced Reconnaissance Framework

# Usage: ./advanced\_recon.sh <domain>

DOMAIN="\$1"

OUTPUT\_DIR="recon\_\${DOMAIN}\_\${date +%Y%m%d\_%H%M%S}"

COMPREHENSIVE\_REPORT="\${OUTPUT\_DIR}/comprehensive\_report.txt"

```
Create output directory
mkdir -p "$OUTPUT_DIR"
```

```
echo "🎯 Advanced Reconnaissance Framework" | tee
"$COMPREHENSIVE_REPORT"
echo "Target: $DOMAIN" | tee -a "$COMPREHENSIVE_REPORT"
echo "Date: $(date)" | tee -a "$COMPREHENSIVE_REPORT"
echo "===== " | tee -a
"$COMPREHENSIVE_REPORT"
```

```
Function to perform comprehensive ping analysis
```

```
advanced_ping_analysis() {
 echo "🔍 Advanced Ping Analysis" | tee -a "$COMPREHENSIVE_REPORT"
 echo "-----" | tee -a "$COMPREHENSIVE_REPORT"
```

```
 local ping_file="$${OUTPUT_DIR}/ping_analysis.txt"
 ping -c 10 "$DOMAIN" > "$ping_file"
```

```
 # Analyze ping statistics
```

```
 local avg_ping=$(grep "avg" "$ping_file" | awk -F'/' '{print $5}')
 local packet_loss=$(grep "packet loss" "$ping_file" | awk '{print $6}')
```

```
 echo "📊 Network Statistics:" | tee -a "$COMPREHENSIVE_REPORT"
 echo " Average RTT: $avg_ping ms" | tee -a "$COMPREHENSIVE_REPORT"
 echo " Packet Loss: $packet_loss" | tee -a "$COMPREHENSIVE_REPORT"
 echo "" | tee -a "$COMPREHENSIVE_REPORT"
}
```

```
Function for comprehensive DNS reconnaissance
```

```
comprehensive_dns_scan() {
 echo "🌐 Comprehensive DNS Analysis" | tee -a
"$COMPREHENSIVE_REPORT"
 echo "-----" | tee -a "$COMPREHENSIVE_REPORT"
```

```
 local dns_file="$${OUTPUT_DIR}/dns_records.txt"
```

```

Get all DNS record types
for record in A AAAA MX TXT NS SOA CNAME; do
 echo " $record Records:" | tee -a "$COMPREHENSIVE_REPORT"
 dig "$DOMAIN" $record +short | tee -a "$COMPREHENSIVE_REPORT" |
tee -a "$dns_file"
 echo "" | tee -a "$COMPREHENSIVE_REPORT"
done

DNS reputation check
echo " DNS Blacklist Check:" | tee -a "$COMPREHENSIVE_REPORT"
local domain_ip=$(dig +short "$DOMAIN" | head -1)
echo "Checking IP $domain_ip against blacklists..." | tee -a
"$COMPREHENSIVE_REPORT"
echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

Function to generate reconnaissance summary
generate_summary() {
 echo " Reconnaissance Summary" | tee -a
"$COMPREHENSIVE_REPORT"
 echo "-----" | tee -a "$COMPREHENSIVE_REPORT"

 local ip_count=$(dig "$DOMAIN" A +short | wc -l)
 local mx_count=$(dig "$DOMAIN" MX +short | wc -l)
 local ns_count=$(dig "$DOMAIN" NS +short | wc -l)

 echo " Infrastructure Overview:" | tee -a "$COMPREHENSIVE_REPORT"
 echo " IP Addresses: $ip_count" | tee -a "$COMPREHENSIVE_REPORT"
 echo " Mail Servers: $mx_count" | tee -a "$COMPREHENSIVE_REPORT"
 echo " Name Servers: $ns_count" | tee -a "$COMPREHENSIVE_REPORT"
 echo "" | tee -a "$COMPREHENSIVE_REPORT"

 echo " Security Observations:" | tee -a "$COMPREHENSIVE_REPORT"
 # Add automated security observations based on findings
 echo " - Check for SPF/DKIM/DMARC records in TXT" | tee -a
"$COMPREHENSIVE_REPORT"
 echo " - Verify name server security" | tee -a "$COMPREHENSIVE_REPORT"
}

```

```

 echo " - Analyze subdomain structure" | tee -a
"$COMPREHENSIVE_REPORT"
 echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

Main execution
main() {
 if [-z "$DOMAIN"]; then
 echo "Usage: $0 <domain>"
 exit 1
 fi

 echo "🚀 Starting comprehensive reconnaissance..." | tee -a
"$COMPREHENSIVE_REPORT"
 echo "" | tee -a "$COMPREHENSIVE_REPORT"

 advanced_ping_analysis
 comprehensive_dns_scan
 generate_summary

 echo "✅ Reconnaissance complete!" | tee -a
"$COMPREHENSIVE_REPORT"
 echo "📁 Results saved in: $OUTPUT_DIR" | tee -a
"$COMPREHENSIVE_REPORT"
 echo "📄 Main report: $COMPREHENSIVE_REPORT" | tee -a
"$COMPREHENSIVE_REPORT"
}

main "$@"
...

Step 2: Run Comprehensive Reconnaissance
```bash
chmod +x advanced_recon.sh
./advanced_recon.sh example.com

```

```
pe here to sea...
My Computer
kali-linux-2025.4-
Metasploitable2-
OWASP Broken W

kali@kali: ~
Session Actions Edit View Help
GNU nano 9.0
#!/bin/bash
# Advanced Reconnaissance Framework
# Usage: ./advanced_recon.sh <domain>

DOMAIN="$1"
OUTPUT_DIR="recon_${DOMAIN}_${date +%Y%m%d_%H%M%S}"
COMPREHENSIVE_REPORT="${OUTPUT_DIR}/comprehensive_report.txt"

# Create output directory
mkdir -p "$OUTPUT_DIR"

echo "🔍 Advanced Reconnaissance Framework" | tee "$COMPREHENSIVE_REPORT"
echo "Target: $DOMAIN" | tee -a "$COMPREHENSIVE_REPORT"
echo "Date: $(date)" | tee -a "$COMPREHENSIVE_REPORT"
echo "===== " | tee -a "$COMPREHENSIVE_REPORT"

# Function to perform comprehensive ping analysis
advanced_ping_analysis() {
    echo "📡 Advanced Ping Analysis" | tee -a "$COMPREHENSIVE_REPORT"
    echo "===== " | tee -a "$COMPREHENSIVE_REPORT"

    local ping_file="${OUTPUT_DIR}/ping_analysis.txt"
    ping -c 10 "$DOMAIN" > "$ping_file"

    # Analyze ping statistics
    local avg_ping=$(grep "avg" "$ping_file" | awk -F '/' '{print $5}')
    local packet_loss=$(grep "packet loss" "$ping_file" | awk '{print $6}')

    echo "📊 Network Statistics:" | tee -a "$COMPREHENSIVE_REPORT"
    echo "   Average RTT: $avg_ping ms" | tee -a "$COMPREHENSIVE_REPORT"
    echo "   Packet Loss: $packet_loss" | tee -a "$COMPREHENSIVE_REPORT"
    echo "" | tee -a "$COMPREHENSIVE_REPORT"
}

# Function for comprehensive DNS reconnaissance
comprehensive_dns_scan() {
    echo "🌐 Comprehensive DNS Analysis" | tee -a "$COMPREHENSIVE_REPORT"
    echo "===== " | tee -a "$COMPREHENSIVE_REPORT"

    local dns_file="${OUTPUT_DIR}/dns_records.txt"

    # Get all DNS record types
    for record in A AAAA MX TXT NS SOA CNAME; do
        echo "📄 $record Records:" | tee -a "$COMPREHENSIVE_REPORT"
    done
}

input to this VM, move the mouse pointer inside or press Ctrl+G
```

```
kali@kali: ~  
$ ./advanced_recon.sh google.com  
Advanced Reconnaissance Framework  
Target: google.com  
Date: Thu May 21 10:09:59 AM EDT 2026  
  
Starting comprehensive reconnaissance ...  
  
Advanced Ping Analysis  
  
Network Statistics:  
Average RTT: 236.148 ms  
Packet Loss: 0%  
  
Comprehensive DNS Analysis  
  
A Records:  
142.251.216.110  
  
AAAA Records:  
2c0f:fb50:4003:803::200e  
  
MX Records:  
10 smtp.google.com.  
  
TXT Records:  
;; communications error to 10.101.8.149#53: timed out  
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"  
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"  
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4o8-0Hf5o"  
"docuSign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"  
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"  
"apple-domain-verification=30afIBcvSuDV2PLX"  
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"  
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"  
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="  
"v=spf1 include:_spf.google.com ~all"  
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamu0XBivAzpR5IZ87D0"  
"docuSign=1b0a6754-49b1-4db5-8540-d2c12664b289"  
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cp0JM0nikft0jAgjmsQ"  
  
NS Records:  
ns1.google.com.  
ns2.google.com.  
ns3.google.com.  
ns4.google.com.
```

Report on Analysis:

The scan showed stable network performance with 0% packet loss and an average RTT of 236 ms. DNS analysis revealed IPv4 and IPv6 addresses, one mail server (smtp.google.com), and four authoritative name servers, indicating a highly redundant infrastructure.

TXT records included multiple domain verification entries and SPF configuration, showing strong email and domain security controls. Overall, the target demonstrates a well-structured and highly secured DNS and network architecture.

Part 6: Professional Reporting and Analysis

Exercise 6.1: Create Reconnaissance Report Template

Lab Deliverables and Assessment

Required Submissions:

1. Completed reconnaissance scripts (`basic\_recon.sh`, `passive\_intel.sh`, `advanced\_recon.sh`)

```
to sea...
puter
linux-2025.4-
sploitable2-
SP Broken W

kali-linux-2025.4-vmware-a...

Session Actions Edit View Help

(kali@kali)-[~]
$ ./basic_recon.sh google.com
Starting Basic Reconnaissance for: google.com

Ping Analysis for google.com

Target is reachable
PING google.com (172.217.22.46) 56(84) bytes of data.
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=1 ttl=110 time=208 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=2 ttl=110 time=146 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=3 ttl=110 time=291 ms
64 bytes from fra15s16-in-f46.1e100.net (172.217.22.46): icmp_seq=4 ttl=110 time=210 ms

— google.com ping statistics —
4 packets transmitted, 4 received, 0% packet loss, time 3010ms
rtt min/avg/max/mdev = 145.790/213.765/291.171/51.624 ms
IP Address: 172.217.22.46

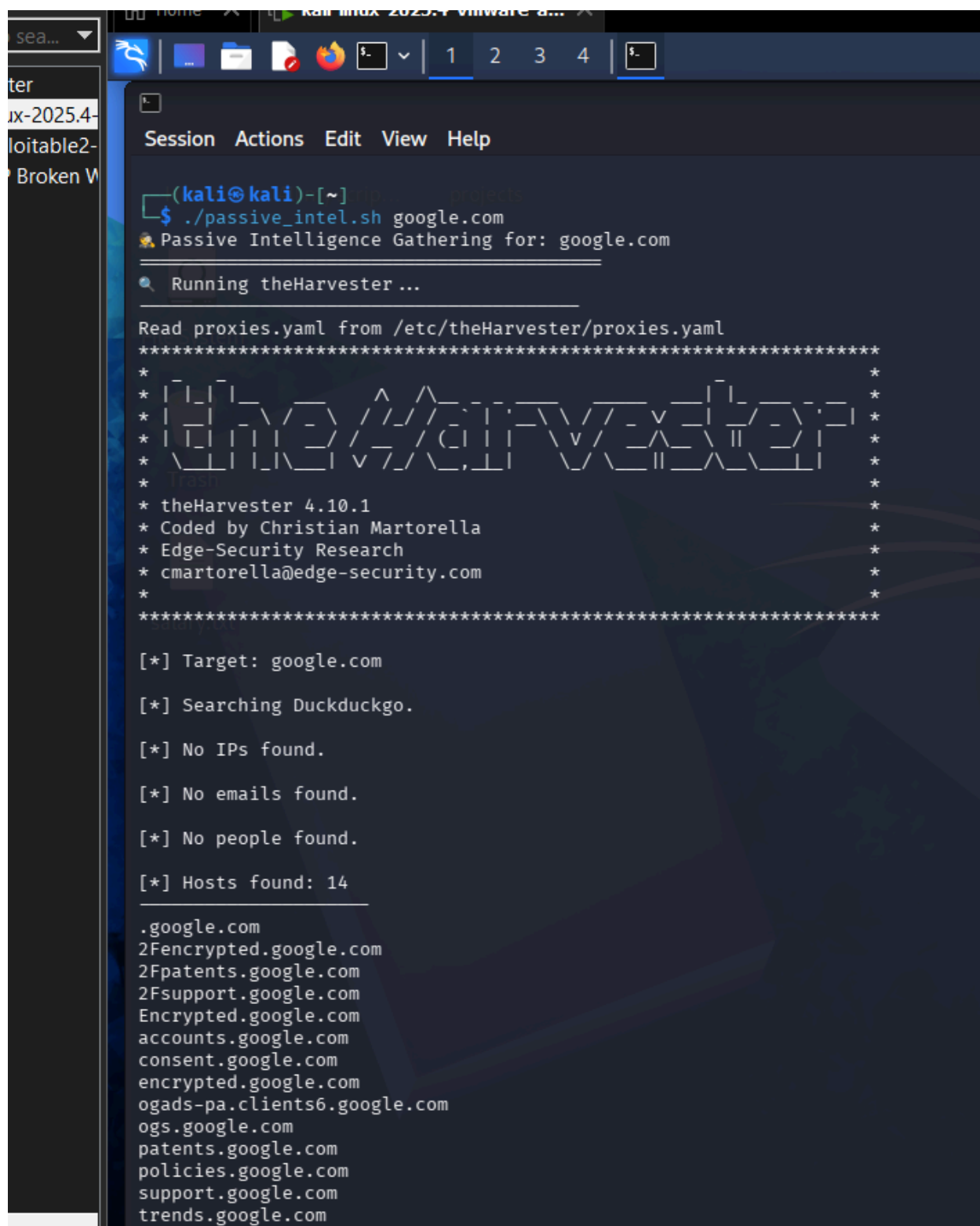
WHOIS Analysis for google.com

Registrar Information:
getaddrinfo(whois.verisign-grs.com): Temporary failure in name resolution

Dates Information:
Updated Date: 2019-09-09T15:39:04Z
Creation Date: 1997-09-15T04:00:00Z
NOTICE: The expiration date displayed in this record is the date the
currently set to expire. This date does not necessarily reflect the expiration
view the registrar's reported date of expiration for this registration.

Contact Information:
Registrar Abuse Contact Email: abusecomplaints@markmonitor.com
Registrant Country: US
Registrant Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
Tech Email: Select Request Email Form at https://domains.markmonitor.com/whois/google.com
contact, and such email address is not visible above, you may do so via our web

Name Servers:
Name Server: NS1.GOOGLE.COM
Name Server: NS2.GOOGLE.COM
Name Server: NS3.GOOGLE.COM
Name Server: NS4.GOOGLE.COM
Name Server: ns2.google.com
Name Server: ns3.google.com
Name Server: ns1.google.com
Name Server: ns4.google.com
```



```
(kali@kali)-[~]
$ ./advanced_recon.sh google.com
🔍 Advanced Reconnaissance Framework
Target: google.com
Date: Thu May 21 01:56:24 PM EDT 2026

🚀 Starting comprehensive reconnaissance ...

🔍 Advanced Ping Analysis

📊 Network Statistics:
Average RTT: 142.243 ms
Packet Loss: 10%

🌐 Comprehensive DNS Analysis

📄 A Records:
142.251.216.110

📄 AAAA Records:
2c0f:fb50:4003:803::200e

📄 MX Records:
;; communications error to 10.101.8.149#53: timed out
10 smtp.google.com.

📄 TXT Records:
;; communications error to 10.101.8.149#53: timed out
"google-site-verification=4ibFUGB-wXLQ_S7vsXVomSTVamuOXBivAZpR5IZ87D0"
"apple-domain-verification=30afIBcvSuDV2PLX"
"facebook-domain-verification=22rm551cu4k0ab0bxsw536tlds4h95"
"onetrust-domain-verification=0d477fe608074e6f9c12bca7826035cc"
"cisco-ci-domain-verification=47c38bc8c4b74b7233e9053220c1bbe76bcc1cd33c7acf7acd36cd6a5332004b"
"v=spf1 include:_spf.google.com ~all"
"google-site-verification=wD8N7i1JTNTkezJ49swvWW48f8_9xveREV4oB-0Hf5o"
"docusign=1b0a6754-49b1-4db5-8540-d2c12664b289"
"MS=E4A68B9AB2BB9670BCE15412F62916164C0B20BB"
"google-site-verification=TV9-DBe4R80X4v0M4U_bd_J9cpOJM0nikft0jAgjmsQ"
"globalsign-smime-dv=CDYX+XFHUw2wml6/Gb8+59BsH31KzUr6c1l2BPvqKX8="
"docusign=05958488-4752-4ef2-95eb-aa7ba8a3bd0e"
"onetrust-domain-verification=6d685f1d41a94696ad7ef771f68993e0"

📄 NS Records:
ns1.google.com.
ns2.google.com.
ns3.google.com.
ns4.google.com.
```

Critical Thinking Questions:

1. Legal and Ethical Considerations:

- What constitutes authorized vs unauthorized reconnaissance?

Authorized reconnaissance is conducted with proper permission from the organization or within an approved scope such as a penetration testing contract

or lab environment. Unauthorized reconnaissance involves scanning or gathering information from systems without consent, which may violate laws and organizational policies.

- How do you ensure your activities remain within legal boundaries?

To remain within legal limits, reconnaissance activities should only target approved domains and systems. Security professionals must follow defined scopes, avoid disruptive actions, document permissions, and comply with cybersecurity laws and ethical guidelines.

2. Technical Analysis:

- What DNS misconfigurations commonly lead to security?

vulnerabilities? Common DNS misconfigurations include exposed zone transfers, missing SPF/DKIM/DMARC records, improperly configured name servers, and outdated DNS entries. These weaknesses can lead to email spoofing, information leakage, or unauthorized access.

- How can TTL values be used in network mapping?

TTL (Time To Live) values help estimate the number of network hops between a source and target system. Different operating systems also use different default TTL values, allowing analysts to infer operating systems and understand network structure during reconnaissance.

3. Strategic Thinking:

- How would you prioritize findings for a penetration test?

High-risk findings such as exposed administrative portals, vulnerable services, weak email protections, and sensitive subdomains should be prioritized first because they may provide direct attack paths into the target environment.

- What additional reconnaissance would you conduct for a comprehensive assessment?

Additional reconnaissance may include port scanning, service enumeration, vulnerability scanning, SSL certificate analysis, technology fingerprinting, employee OSINT gathering, and web application mapping to achieve a more complete security assessment.

Recommendations

1. Regularly audit and remove unused or outdated subdomains to reduce attack surface.
2. Enforce full email security policies including SPF, DKIM, and DMARC.
3. Monitor DNS records and certificate transparency logs for unauthorized changes.
4. Implement continuous security monitoring for exposed services.
5. Restrict unnecessary public exposure of internal systems where possible.

Conclusion

The reconnaissance process successfully demonstrated how attackers or security analysts can gather critical information without direct access to systems. Domains analyzed showed strong infrastructure security, but also a large attack surface due to numerous subdomains and exposed public services.

Tools like `dig`, `nslookup`, and `theHarvester` proved essential for mapping DNS records, identifying mail servers, and discovering subdomains.

Lab 2: Website Enumeration and Information Gathering

Tools Used

- Kali Linux
- Whatweb

Exercise 1:

whatweb command to detect technologies for the following targets:

- example.com
- stackoverflow.com
- github.com

Command Used: whatweb example.com, whatweb stackoverflow.com, whatweb github.com

```

(kali@kali)-[~]
$ whatweb example.com
http://example.com [200 OK] Allow[GET, HEAD], Country[RESERVED][22], HTML5, HTTPServer[cloudflare], IP[172.66.147.243], Title[Example Domain], UncommonHeaders[cf-cache-status,cf-ray]
https://example.com [200 OK] Allow[GET, HEAD], Country[UNITED STATES][US], HTML5, HTTPServer[cloudflare], IP[104.20.23.154], Title[Example Domain], UncommonHeaders[cf-cache-status,cf-ray]

(kali@kali)-[~]
$ whatweb stackoverflow.com
http://stackoverflow.com [301 Moved Permanently] Cookies[_cf_bm], Country[UNITED STATES][US], HTTPServer[cloudflare], HttpOnly[_cf_bm], IP[198.252.206.1], RedirectLocation[https://stackoverflow.com/], Title[301 Moved Permanently], UncommonHeaders[x-dns-prefetch-control,cf-ray]
https://stackoverflow.com [403 Forbidden] Cookies[_cf_bm], Country[UNITED STATES][US], HTML5, HTTPServer[cloudflare], HttpOnly[_cf_bm], IP[198.252.206.1], Script, Title[Just a moment ...], UncommonHeaders[accept-ch,cf-mitigated,content-security-policy,critical-ch,cross-origin-embedder-policy,cross-origin-opener-policy,cross-origin-resource-policy,origin-agent-cluster,permissions-policy,referrer-policy,server-timing,x-content-type-options,x-dns-prefetch-control,cf-ray], X-Frame-Options[SAMEORIGIN], X-UA-Compatible[IE=Edge]
https://stackoverflow.com/ [403 Forbidden] Country[UNITED STATES][US], HTML5, HTTPServer[cloudflare], IP[198.252.206.1], Script, Title[Just a moment ...], UncommonHeaders[accept-ch,cf-mitigated,content-security-policy,critical-ch,cross-origin-embedder-policy,cross-origin-opener-policy,cross-origin-resource-policy,origin-agent-cluster,permissions-policy,referrer-policy,server-timing,x-content-type-options,x-dns-prefetch-control,cf-ray], X-Frame-Options[SAMEORIGIN], X-UA-Compatible[IE=Edge]

(kali@kali)-[~]
$ whatweb github.com
http://github.com [301 Moved Permanently] Country[UNITED STATES][US], IP[140.82.121.4], RedirectLocation[https://github.com/]
https://github.com [200 OK] Content-Language[en-US], Cookies[_gh_sess,octo,logged_in], Country[UNITED STATES][US], Email[you@domain.com], HTML5, HTTPServer[github.com], HttpOnly[_gh_sess,logged_in], IP[140.82.121.4], Open-Graph-Protocol[object][1401488693436528], OpenSearch[opensearch.xml], Script[application/javascript,application/json], Strict-Transport-Security[max-age=31536000; includeSubdomains; preload], Title[GitHub · Change is constant. GitHub keeps you ahead. · GitHub], UncommonHeaders[x-content-type-options,referrer-policy,content-security-policy,x-github-request-id], X-Frame-Options[deny], X-XSS-Protection[0]
https://github.com/ [200 OK] Content-Language[en-US], Cookies[_gh_sess,octo,logged_in], Country[UNITED STATES][US], Email[you@domain.com], HTML5, HTTPServer[github.com], HttpOnly[_gh_sess,logged_in], IP[140.82.121.4], Open-Graph-Protocol[object][1401488693436528], OpenSearch[opensearch.xml], Script[application/javascript,application/json], Strict-Transport-Security[max-age=31536000; includeSubdomains; preload], Title[GitHub · Change is constant. GitHub keeps you ahead. · GitHub], UncommonHeaders[x-content-type-options,referrer-policy,content-security-policy,x-github-request-id], X-Frame-Options[deny], X-XSS-Protection[0]

```

Record of Findings:

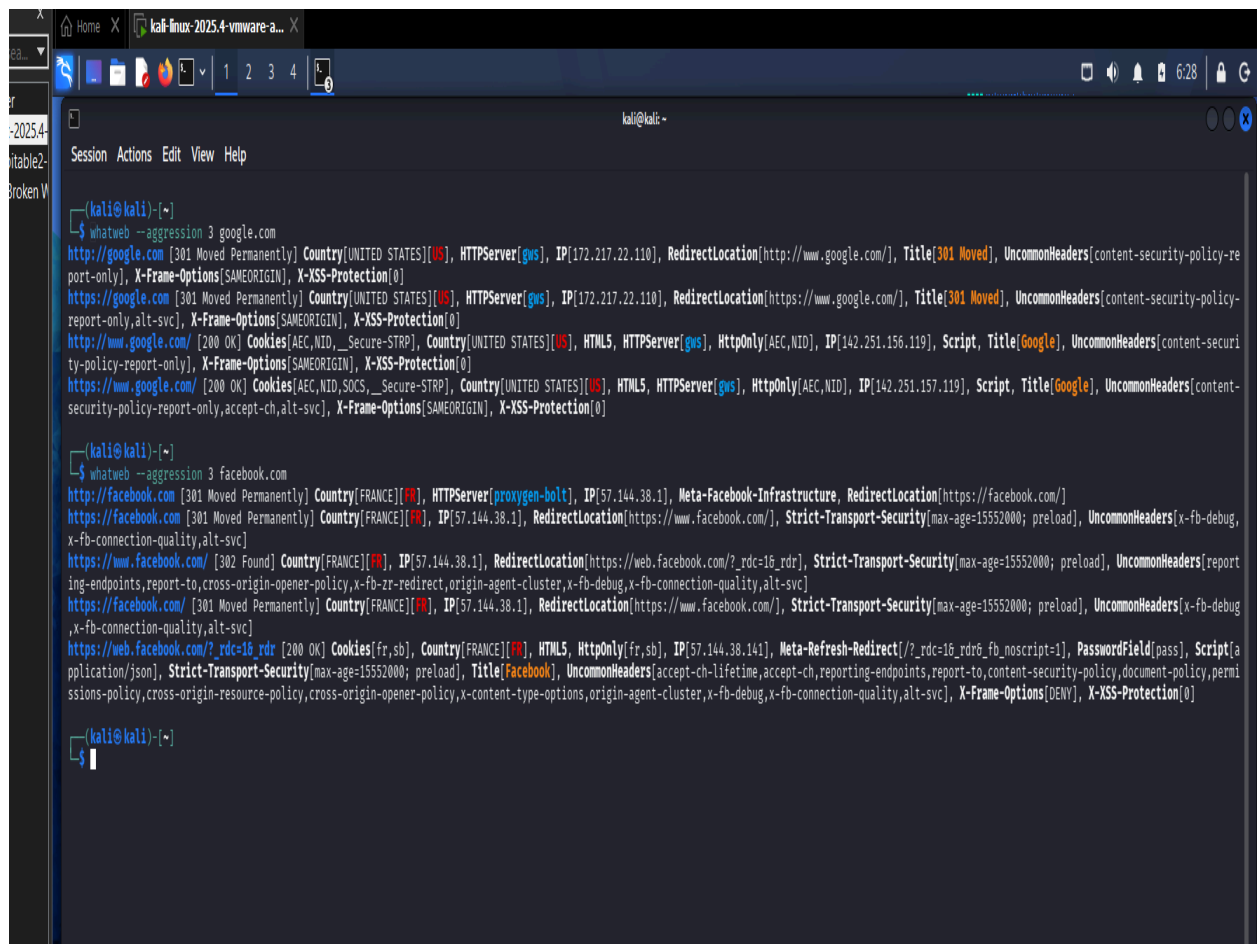
1. example.com: HTML5 and Cloudflare
2. stackoverflow.com: HTML5,Cloudflare,Script
3. github.com: HTML5,JavaScript, JSON,Open Graph Protocol, OpenSearch

Exercise 2:

Aggressive scan on the following targets:

- google.com
- facebook.com

Commands Used: whatweb --aggression 3 [google.com](https://www.google.com/), whatweb --aggression 3 [facebook.com](https://www.facebook.com/).



```
(kali@kali)~$ whatweb --aggression 3 google.com
http://google.com [301 Moved Permanently] Country[UNITED STATES][US], HTTPServer[gws], IP[172.217.22.110], RedirectLocation[http://www.google.com/], Title[301 Moved], UncommonHeaders[content-security-policy-report-only], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]
https://google.com [301 Moved Permanently] Country[UNITED STATES][US], HTTPServer[gws], IP[172.217.22.110], RedirectLocation[https://www.google.com/], Title[301 Moved], UncommonHeaders[content-security-policy-report-only,alt-svc], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]
http://www.google.com/ [200 OK] Cookies[AEC,NID,_Secure-STRP], Country[UNITED STATES][US], HTML5, HTTPServer[gws], HttpOnly[AEC,NID], IP[142.251.156.119], Script, Title[Google], UncommonHeaders[content-security-policy-report-only], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]
https://www.google.com/ [200 OK] Cookies[AEC,NID,SOCS,_Secure-STRP], Country[UNITED STATES][US], HTML5, HTTPServer[gws], HttpOnly[AEC,NID], IP[142.251.157.119], Script, Title[Google], UncommonHeaders[content-security-policy-report-only,accept-ch,alt-svc], X-Frame-Options[SAMEORIGIN], X-XSS-Protection[0]

(kali@kali)~$ whatweb --aggression 3 facebook.com
http://facebook.com [301 Moved Permanently] Country[FRANCE][FR], HTTPServer[proxygen-bolt], IP[57.144.38.1], Meta-Facebook-Infrastructure, RedirectLocation[https://facebook.com/]
https://facebook.com [301 Moved Permanently] Country[FRANCE][FR], IP[57.144.38.1], RedirectLocation[https://www.facebook.com/], Strict-Transport-Security[max-age=15552000; preload], UncommonHeaders[x-fb-debug,x-fb-connection-quality,alt-svc]
https://www.facebook.com/ [302 Found] Country[FRANCE][FR], IP[57.144.38.1], RedirectLocation[https://web.facebook.com/?_rdc=1b_rdr], Strict-Transport-Security[max-age=15552000; preload], UncommonHeaders[reporting-endpoints,report-to,cross-origin-opener-policy,x-fb-zr-redirect,origin-agent-cluster,x-fb-debug,x-fb-connection-quality,alt-svc]
https://facebook.com/?_rdc=1b_rdr [301 Moved Permanently] Country[FRANCE][FR], IP[57.144.38.1], RedirectLocation[https://www.facebook.com/], Strict-Transport-Security[max-age=15552000; preload], UncommonHeaders[x-fb-debug,x-fb-connection-quality,alt-svc]
https://web.facebook.com/?_rdc=1b_rdr [200 OK] Cookies[fr,sb], Country[FRANCE][FR], HTML5, HttpOnly[fr,sb], IP[57.144.38.141], Meta-Refresh-Redirect[?_rdc=1b_rdr&fb_noscript=1], PasswordField[pass], Script[application/json], Strict-Transport-Security[max-age=15552000; preload], Title[Facebook], UncommonHeaders[accept-ch,reporting-endpoints,report-to,content-security-policy,document-policy,permissions-policy,cross-origin-resource-policy,cross-origin-opener-policy,x-content-type-options,origin-agent-cluster,x-fb-debug,x-fb-connection-quality,alt-svc], X-Frame-Options[DENY], X-XSS-Protection[0]
```

Findings:

1. google.com: HTML5 and Script
2. facebook.com: HTML5, Script, JSON, Meta-Facebook-Infrastructure, Meta-Refresh-Redirect

Lab 3: Subdomain Hunting

Tools Used

- Kali Linux
- sublist3r
- dirb
- KnockPy

Exercise 1:

sublist3r command for the following domains:

- github.com
- google.com

Findings:

1. Subdomains for github.com:
135 unique subdomains were discovered.
2. Subdomains for google.com:
180 unique subdomains were discovered.

Exercise 2:

Directory discovery scan on the following targets:

Commands used:

- dirb http://example.com`
- dirb <http://example.org>`

```

kali@kali: [~]
(kali@kali)-[~]
$ dirb https://example.com

DIRB v2.22
By The Dark Raver
Threads: 250

START_TIME: Fri May 22 07:39:19 2026
URL_BASE: https://example.com/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt
Reports DB: ~/knockpy/reports.db
API Keys: False

GENERATED WORDS: 4612

Scanning URL: https://example.com/

END_TIME: Fri May 22 07:58:02 2026
DOWNLOADED: 4612 - FOUND: 0

(kali@kali)-[~]
$

Scanning domains
1/1 000000

Domain IP

```

```
Session Actions Edit View Help
(kaliⓈkali)-[~]
$ dirb http://example.org

DIRB v2.22
By The Dark Raver
Timeout: 3.0
START_TIME: Fri May 22 07:52:45 2026
URL_BASE: http://example.org/
WORDLIST_FILES: /usr/share/dirb/wordlists/common.txt ; x64) AppleWeb
Wordlist: /usr/lib/python3/dist-packages/knockpy/wordlist/wo
~/.knockpy/reports.db
Silent: False
GENERATED WORDS: 4612

Scanning URL: http://example.org/

END_TIME: Fri May 22 08:09:17 2026
DOWNLOADED: 4612 - FOUND: 0

(kaliⓈkali)-[~]
$
```

Findings:

- 1. Directories for example.com:
 - dirb tested 4612 possible directories/files but found nothing on <https://example.com>
- 2. Directories for example.org
 - dirb tested 4612 possible directories/files but found nothing on <https://example.com>

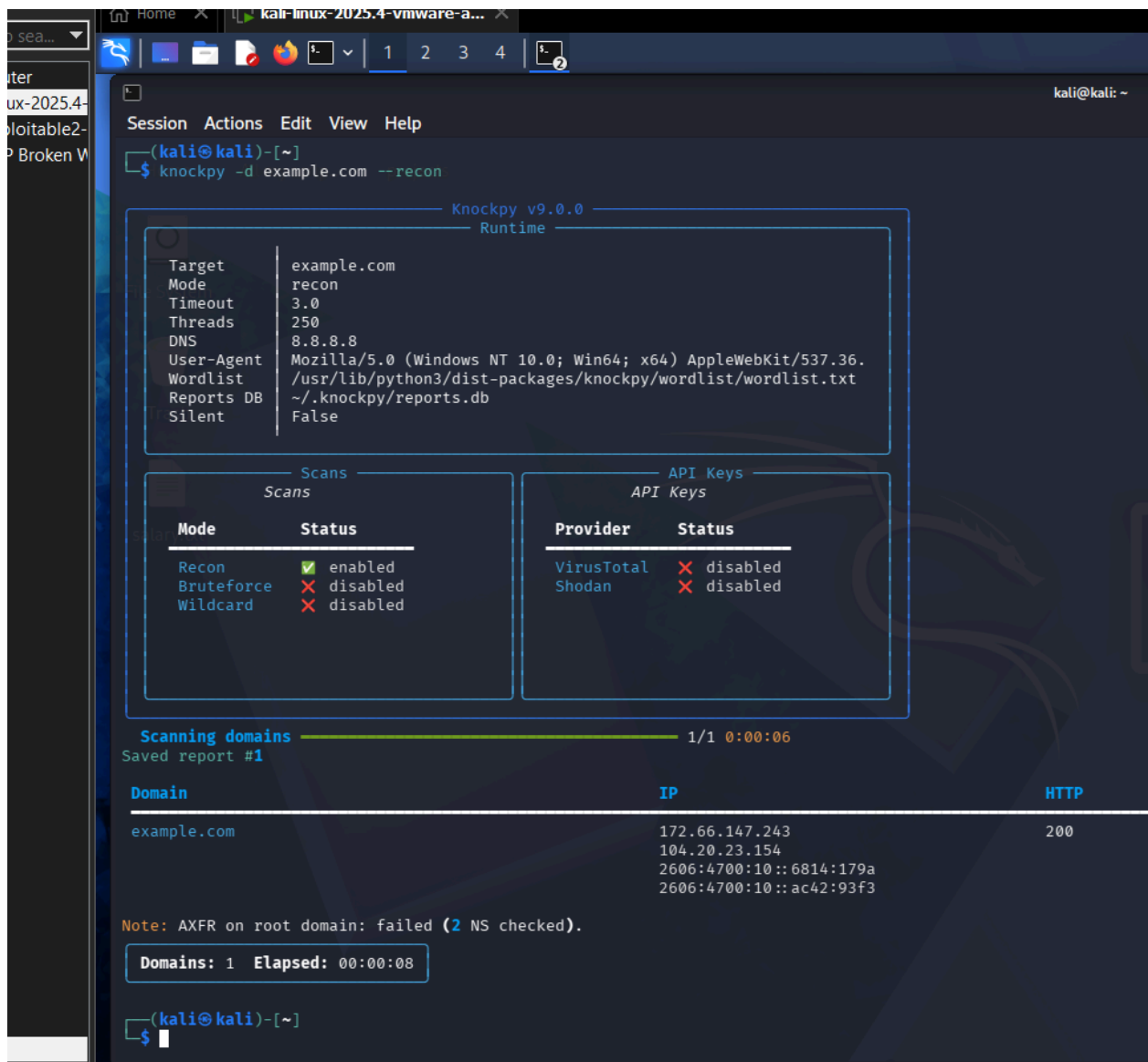
Exercise 3

Used KnockPy to enumerate subdomains for the following domain:

Target Domain: example.com

Commands used:

Knockpy -d example.com --recon



```
(kali@kali)-[~]
$ knockpy -d example.com --recon

Knockpy v9.0.0
Runtime
-----
Target      example.com
Mode        recon
Timeout     3.0
Threads     250
DNS         8.8.8.8
User-Agent  Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36.
Wordlist     /usr/lib/python3/dist-packages/knockpy/wordlist/wordlist.txt
Reports DB  ~/.knockpy/reports.db
Silent      False

Scans
-----
Mode      Status
Recon     ✓ enabled
Bruteforce ✗ disabled
Wildcard  ✗ disabled

API Keys
-----
Provider  Status
VirusTotal ✗ disabled
Shodan    ✗ disabled

Scanning domains 1/1 0:00:06
Saved report #1

Domain      IP                        HTTP
example.com 172.66.147.243
             104.20.23.154
             2606:4700:10::6814:179a
             2606:4700:10::ac42:93f3

Note: AXFR on root domain: failed (2 NS checked).

Domains: 1 Elapsed: 00:00:08

(kali@kali)-[~]
$
```

Findings

Discovered Subdomains and Information:

KnockPy checked subdomains for `example.com`

- It found no subdomains
- It tried DNS zone transfer (AXFR) but failed

Lab 4: Basic Port Scanning

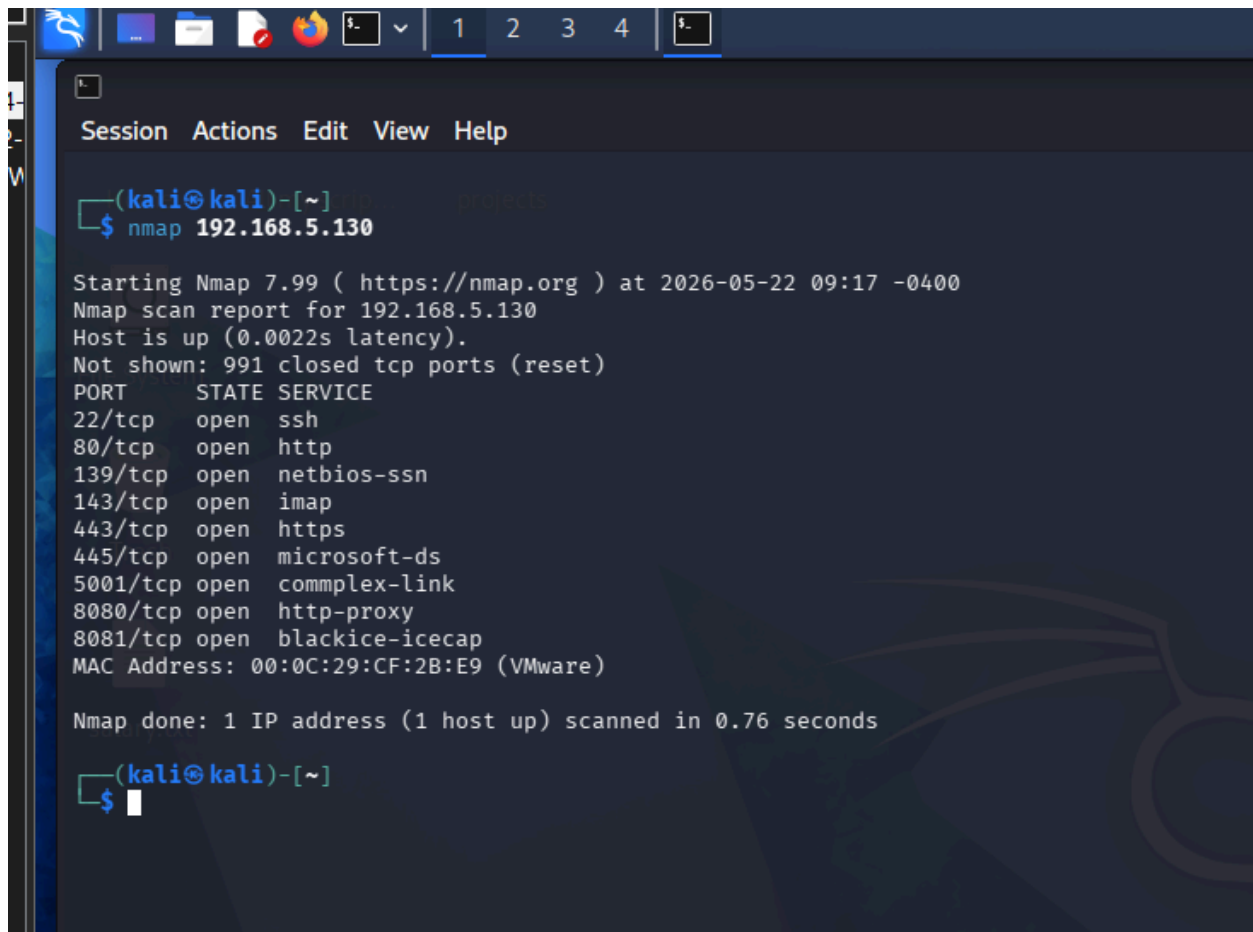
Tools Used

- Kali Linux
- nmap
- Nikto

Exercise 1:

Perform a basic port scan on your OWASP VM IP address and record

Command Used: `nmap 192.168.5.130`



```
(kali@kali)-[~]
$ nmap 192.168.5.130

Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 09:17 -0400
Nmap scan report for 192.168.5.130
Host is up (0.0022s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
139/tcp    open  netbios-ssn
143/tcp    open  imap
443/tcp    open  https
445/tcp    open  microsoft-ds
5001/tcp   open  complex-link
8080/tcp   open  http-proxy
8081/tcp   open  blackice-icecap
MAC Address: 00:0C:29:CF:2B:E9 (VMware)

Nmap done: 1 IP address (1 host up) scanned in 0.76 seconds

(kali@kali)-[~]
$
```

findings:

- Open Ports: 8080,22,80,139,143,443,445,5001 and port 8081

Exercise 2:

Perform an aggressive scan on your OWASP VM IP address

Command used: nmap -sV -O 192.168.5.130

```
kali@kali:~$ nmap -sV -O 192.168.5.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 09:24 -0400
Nmap scan report for 192.168.5.130
Host is up (0.0018s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
22/tcp    open  ssh          OpenSSH 5.3p1 Debian 3ubuntu4 (Ubuntu Linux; protocol 2.0)
80/tcp    open  http         Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
143/tcp   open  imap         Courier Imapd (released 2008)
443/tcp   open  ssl/http     Apache httpd 2.2.14 ((Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL ...)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
5001/tcp  open  java-object  Java Object Serialization
8080/tcp  open  http         Apache Tomcat/Coyote JSP engine 1.1
8081/tcp  open  http         Jetty 6.1.25
1 service unrecognized despite returning data. If you know the service/version, please submit the following fingerprint at https://nmap.org/cgi-bin/submit.cgi?new-service :
SF-Port5001-TCP:V=7.99%I=7%O=5/22%Time=6A105928%P=x86_64-pc-linux-gnu%r(NU
SF:LL,4,"xac\xed\0\x05");
MAC Address: 00:0C:29:CF:2B:E9 (VMware)
Device type: general purpose
Running: Linux 2.6.X
OS CPE: cpe:/o:linux:linux_kernel:2.6
OS details: Linux 2.6.17 - 2.6.36
Network Distance: 1 hop
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 14.42 seconds
```

findings:

Service Versions:

- OpenSSH 5.3p1 Debian 3ubuntu4
- Apache httpd 2.2.14
- Samba smbd 3.x – 4.x
- Courier IMAP (2008 release)
- Apache httpd 2.2.14 (SSL)
- Samba smbd 3.x – 4.x
- Java Object Serialization (port 5001)
- Apache Tomcat/Coyote JSP engine 1.1

- Jetty 6.1.25

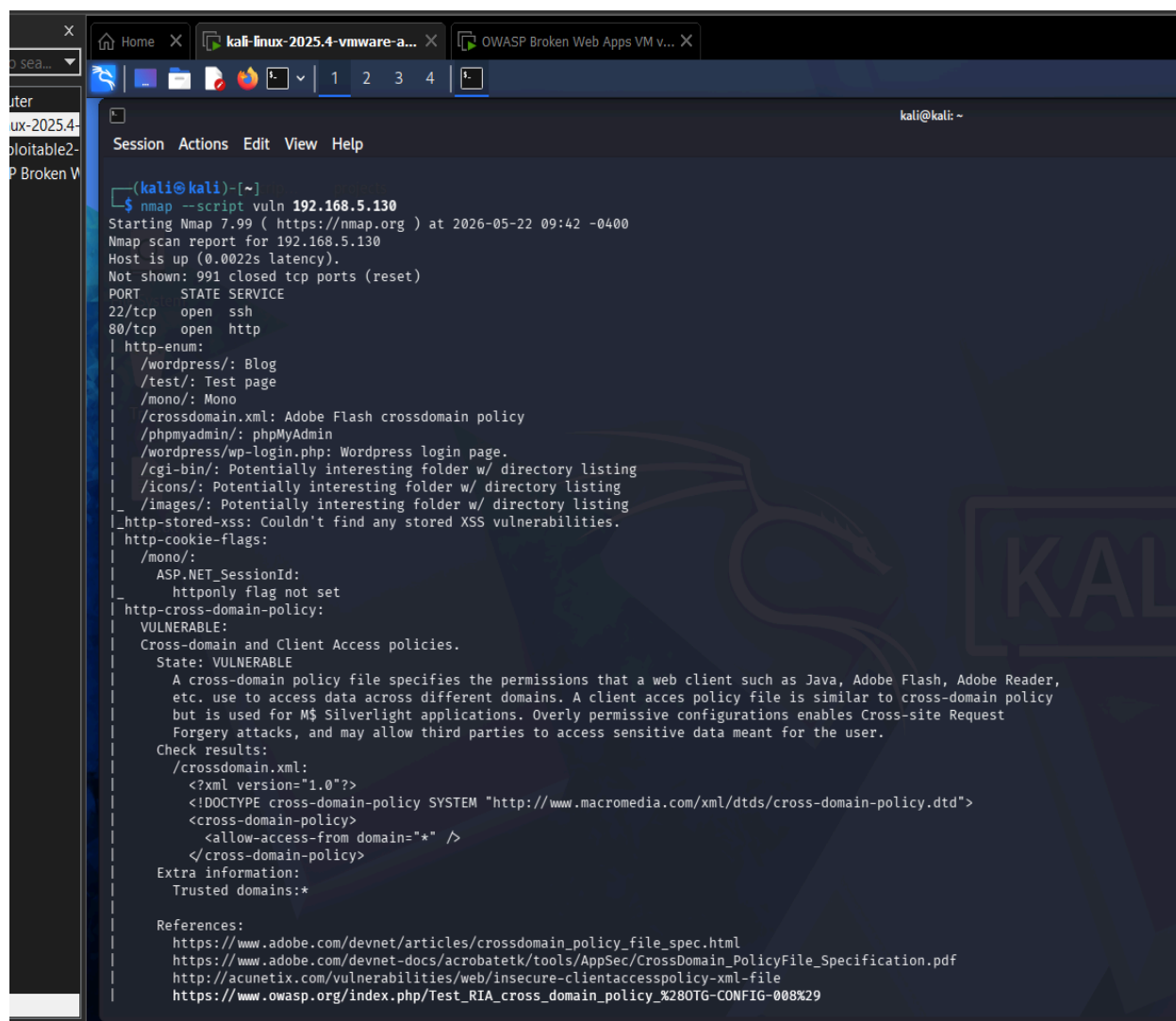
Operating System:

- Linux 2.6.17 – 2.6.36 (Ubuntu Linux)
- VMware virtual machine

Exercise 3:

Conduct a vulnerability scan on your OWASP VM IP address

Commands Used: `nmap --script vuln 192.168.5.130`



```
(kali@kali)-[~]
$ nmap --script vuln 192.168.5.130
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-22 09:42 -0400
Nmap scan report for 192.168.5.130
Host is up (0.0022s latency).
Not shown: 991 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
| http-enum:
| /wordpress/: Blog
| /test/: Test page
| /mono/: Mono
| /crossdomain.xml: Adobe Flash crossdomain policy
| /phpmyadmin/: phpMyAdmin
| /wordpress/wp-login.php: Wordpress login page.
| /cgi-bin/: Potentially interesting folder w/ directory listing
| /icons/: Potentially interesting folder w/ directory listing
| /images/: Potentially interesting folder w/ directory listing
|_ http-stored-xss: Couldn't find any stored XSS vulnerabilities.
| http-cookie-flags:
|   /mono/:
|     ASP.NET_SessionId:
|       httponly flag not set
|_ http-cross-domain-policy:
|   VULNERABLE:
|     Cross-domain and Client Access policies.
|     State: VULNERABLE
|       A cross-domain policy file specifies the permissions that a web client such as Java, Adobe Flash, Adobe Reader,
|       etc. use to access data across different domains. A client access policy file is similar to cross-domain policy
|       but is used for M$ Silverlight applications. Overly permissive configurations enables Cross-site Request
|       Forgery attacks, and may allow third parties to access sensitive data meant for the user.
|     Check results:
|       /crossdomain.xml:
|       <?xml version="1.0"?>
|       <!DOCTYPE cross-domain-policy SYSTEM "http://www.macromedia.com/xml/dtds/cross-domain-policy.dtd">
|       <cross-domain-policy>
|         <allow-access-from domain="*" />
|       </cross-domain-policy>
|     Extra information:
|       Trusted domains:*
|
| References:
|   https://www.adobe.com/devnet/articles/crossdomain_policy_file_spec.html
|   https://www.adobe.com/devnet-docs/acrobatetk/tools/AppSec/CrossDomain_PolicyFile_Specification.pdf
|   http://acunetix.com/vulnerabilities/web/insecure-clientaccesspolicy-xml-file
|   https://www.owasp.org/index.php/Test_RIA_cross_domain_policy_%28OTG-CONFIG-008%29
```

findings:

Vulnerabilities:

- Cross-domain and Client Access policy misconfiguration (/crossdomain.xml)
- Apache byterange filter DoS (CVE-2011-3192)
- HTTP TRACE method enabled
- Internal IP disclosure (127.0.1.1)
- CSRF vulnerabilities (multiple forms on WordPress, Rails Goat, phpBB2, etc.)
- SSL/TLS CCS Injection (CVE-2014-0224)
- Weak Diffie-Hellman group (1024-bit MODP)

- SSL POODLE vulnerability (CVE-2014-3566)
 - Missing HTTPOnly cookie flag (ASP.NET_SessionId)
 - Missing Secure cookie flag (HTTPS session cookies)
 - Directory listing enabled (/cgi-bin/, /icons/, /images/)
 - phpMyAdmin exposed (/phpmyadmin/)
 - WordPress login exposed (/wordpress/wp-login.php)
 - Apache Tomcat admin endpoints exposed (/manager/html, /upload)
 - Potential SQL injection script failure (http-sql-injection script error reported)
-

Exercise 4:

Vulnerability scan on OWASP VM and record your

Commands Used:

1. nikto -h 192.168.5.130

```
kali@kali:~$ nikto -h 192.168.5.130
- Nikto v2.6.0

+ Target IP: 192.168.5.130
+ Target Hostname: 192.168.5.130
+ Target Port: 80
+ Platform: Unknown
+ Start Time: 2026-05-22 09:53:32 (GMT-4)

+ Server: Apache/2.2.14 (Ubuntu) mod_mono/2.4.3 PHP/5.3.2-1ubuntu4.30 with Suhosin-Patch proxy_html/3.0.1 mod_python/3.3.1 Python/2.6.5 mod_ssl/2.2.14 OpenSSL/0.9.8k Phusion_Passenger/4.0.38 mod_perl/2.0.4 Perl/v5.10.1
+ ERROR: Failed to check for updates: 403
+ [999984] /: Server may leak inodes via ETags, header found with file /, inode: 286483, size: 28067, mtime: Thu Jul 30 22:55:52 2015. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2003-1418
+ [750500] /icons/: Directory indexing found.
+ [750500] /cgi-bin/: Directory indexing found.
+ No CGI Directories found (use '-C all' to force check all possible dirs). CGI tests skipped.
+ [500670] /favicon.ico: identifies this app/server as: owasp.org. See: https://en.wikipedia.org/wiki/Favicon
+ [600625] PHP/5.3.2-1ubuntu4.30 appears to be outdated (current is at least 8.5.1).
+ [600595] OpenSSL/0.9.8k appears to be outdated (current is at least 3.6.0). OpenSSL 1.1.1w is current for 1.x and is supported via contract, and 3.0.12 for 3.0.x, and 3.1.4 for 3.1.x.
+ [600620] proxy_html/3.0.1 appears to be outdated (current is at least 3.1.2).
+ [600620] Perl/v5.10.1 appears to be outdated (current is at least v5.36.3).
+ [600511] mod_ssl/2.2.14 appears to be outdated (current is at least 2.9.6) (may depend on server version).
+ [600475] mod_mono/2.4.3 appears to be outdated (current is at least 3.13).
+ [600932] Phusion_Passenger/4.0.38 appears to be outdated (current is at least 6.0.14).
+ [600050] Apache/2.2.14 appears to be outdated (current is at least 2.4.66).
+ [600652] Python/2.6.5 appears to be outdated (current is at least 3.13.1).
+ [600487] mod_perl/2.0.4 appears to be outdated (current is at least 2.0.13).
+ [600494] mod_python/3.3.1 appears to be outdated (current is at least 3.5.0).
+ [999979] /images: IP address found in the 'location' header. The IP is '127.0.1.1'. See: https://portswigger.net/kb/issues/00600300_private-ip-addresses-disclosed
+ [999988] /images: The web server may reveal its internal or real IP in the Location header via a request to with HTTP/1.0. The value is '127.0.1.1'. See: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2000-0649
+ [013587] /: Suggested security header missing: strict-transport-security. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Strict-Transport-Security
+ [013587] /: Suggested security header missing: referrer-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Referrer-Policy
+ [013587] /: Suggested security header missing: content-security-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/CSP
+ [013587] /: Suggested security header missing: x-content-type-options. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/X-Content-Type-Options
+ [013587] /: Suggested security header missing: permissions-policy. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Permissions-Policy
+ [999100] /index: Uncommon header(s) 'tcn' found, with contents: List.
+ [999965] /index: Apache mod_negotiation is enabled with MultiViews, which allows attackers to easily brute force file names. The following alternatives for 'index' were found: index.css, index.html. See: https://www.wisec.it/sectou.php?id=4698bdc59d15,https://exchange.xforce.ibmcloud.com/vulnerabilities/8275
+ [800132] /: mod_ssl/2.2.14 OpenSSL/0.9.8k Phusion_Passenger/4.0.38 mod_perl/2.0.4 Perl/v5.10.1 - mod_ssl 2.8.7 and lower are vulnerable to a remote buffer overflow which may allow a remote shell.
+ [800262] /: PHP/5.3 - PHP 3/4/5 and 7.0 are End of Life products without support.
+ [999990] OPTIONS: Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, TRACE
+ [000434] /: HTTP TRACE method is active and replies which suggests the host is vulnerable to XST. See: https://owasp.org/www-community/attacks/Cross_Site_Tracing
+ [999986] /phpBB2/search.php?search_id=1\\: Retrieved x-powered-by header: PHP/5.3.2-1ubuntu4.30.
+ [95] /phpBB2/search.php?search_id=1\\: Cookie phpbb2owaspbwa_data created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
+ [95] /phpBB2/search.php?search_id=1\\: Cookie phpbb2owaspbwa_sid created without the httponly flag. See: https://developer.mozilla.org/en-US/docs/Web/HTTP/Cookies
```

findings:

Vulnerabilities Found:

- ETags inode leakage (CVE-2003-1418)
- Directory indexing enabled (/icons/, /cgi-bin/, /test/, /images/)

- Outdated Apache 2.2.14
 - Outdated PHP 5.3.2 (EOL)
 - Outdated OpenSSL 0.9.8k (EOL)
 - Outdated Python 2.6.5 (EOL)
 - Outdated Perl 5.10.1 (EOL)
 - Outdated mod_ssl 2.2.14 (possible RCE risk)
 - Outdated mod_mono 2.4.3
 - Outdated Phusion Passenger 4.0.38
 - Outdated mod_perl 2.0.4
 - Outdated mod_python 3.3.1
 - Internal IP disclosure (127.0.1.1 via Location header)
 - Missing security headers:
 - HSTS
 - CSP
 - X-Content-Type-Options
 - Referrer-Policy
 - Permissions-Policy
 - mod_negotiation MultiViews enabled (file brute-force risk)
 - HTTP TRACE method enabled (XST risk)
 - OPTIONS method enabled (GET, POST, TRACE, OPTIONS)
 - mod_ssl buffer overflow risk (legacy versions)
 - phpMyAdmin exposed (/phpmyadmin/)
 - WordPress login exposed (/wordpress/wp-login/)
 - phpBB cookies missing HttpOnly flag
 - Apache default file exposed (/icons/README)
 - Potential sensitive test directory exposed (/test/)
-

Lab 5: Wireshark

Executive Summary

This report presents a network traffic analysis conducted using Wireshark and tshark on a vulnerable OWASP Metasploitable environment. The assessment focused on examining TCP/IP behavior, packet flags, and exposed network services. Key findings include successful TCP 3-way handshakes, exposure of open ports such as SSH (22), HTTP (80), and NetBIOS/SMB (139), and visibility of unencrypted HTTP traffic. The analysis highlights several security risks including exposed services and insecure communication protocols. Recommendations are provided to improve network security posture through encryption, service hardening, and network monitoring.

Objectives

- Capture and analyze network traffic using Wireshark/tshark.
- Identify different types of traffic (TCP, HTTP, HTTPS, etc.).
- Examine packet headers and TCP flags.
- Detect open services and potential vulnerabilities.
- Evaluate overall network security posture.

Methods Used

- **Tool used:** Wireshark / tshark
- Interface monitored: eth0
- Target system: OWASP Metasploitable (vulnerable Linux VM)

Capture Approach:

- Full packet capture (no initial filtering)
- Protocol-based filtering (TCP, HTTP analysis)

Focus Areas:

- TCP handshake behavior
- Service communication on exposed ports
- HTTP traffic inspection

Exercise 1:

Explore the Wireshark GUI. Identify and list the main components you see, including where to find the Statistics menu.

- The **Main component** in the GUI is the Menu Bar which contains: File, Edit, View, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help.
- **Main Toolbar** Located below the Menu Bar contains: Quick-access icons for starting/stopping captures, opening files, saving captures, and applying filters.
- **Display Filter Bar** Located below the toolbar. Used to filter packets using expressions.

- **The Statistics** menu can be found at the very top of the window.(Menu Bar)

Exercise 2:

- Capture network traffic using both Wireshark and tshark. Compare the two methods and note any differences in the user experience.

Screenshots of tshark and wireshark

tshark (Terminal)

- Command-line version of Wireshark
- Shows raw text output of packets
- Faster and lighter (no GUI)
- Requires commands and filters to analyze

Wireshark (GUI)

- Visual interface with colors and graphs
- Easy to click packets and inspect details
- Shows packet flow in layers (Ethernet → IP → TCP → HTTP)
- You can filter using buttons and display filters,
Example:one can click the HTTP packet.

Exercise 3:

- Use filters to analyze different types of traffic. Record the following:
 - The Number of HTTP packets captured: 6 packets.

No.	Source	Destination	Protocol	Length	Info
36	192.168.5.128	192.168.5.2	DNS	97	Standard query 0xe50a A firefox.settings.services.mozilla.com
37	192.168.5.128	192.168.5.2	DNS	109	Standard query 0xe50a A firefox.settings.services.mozilla.com.localdomain
42	192.168.5.2	192.168.5.128	DNS	149	Standard query response 0xe50a A firefox.settings.services.mozilla.com CNAME mozilla.map.fastly.net A 146.75...
43	192.168.5.128	192.168.5.2	ICMP	177	Destination unreachable (Port unreachable)
44	192.168.5.2	192.168.5.128	DNS	161	Standard query response 0x984e A firefox.settings.services.mozilla.com.localdomain CNAME mozilla.map.fastly...
45	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x5c70 AAAA firefox.settings.services.mozilla.com.localdomain
46	192.168.5.128	192.168.5.2	DNS	87	Standard query 0x4a75 A safebrowsing.googleapis.com
50	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x984e A firefox.settings.services.mozilla.com.localdomain
51	192.168.5.2	192.168.5.128	DNS	161	Standard query response 0x984e A firefox.settings.services.mozilla.com.localdomain CNAME mozilla.map.fastly...
52	192.168.5.128	192.168.5.2	DNS	109	Standard query 0x5c70 AAAA firefox.settings.services.mozilla.com.localdomain
55	192.168.5.128	192.168.5.2	DNS	76	Standard query 0xb374 HTTPS learn.uchent.com
56	192.168.5.128	192.168.5.2	DNS	76	Standard query 0x699e A learn.uchent.com
57	192.168.5.128	192.168.5.2	DNS	76	Standard query 0x659d AAAA learn.uchent.com
58	192.168.5.128	192.168.5.2	DNS	74	Standard query 0x7a67 A www.google.com
59	192.168.5.128	192.168.5.2	DNS	74	Standard query 0xde65 AAAA www.google.com
60	192.168.5.2	192.168.5.128	DNS	282	Standard query response 0x7a67 A www.google.com A 142.251.157.119 A 142.251.155.119 A 142.251.156.119 A 142...
61	192.168.5.128	192.168.5.2	DNS	74	Standard query 0xe292 A www.google.com

Frame	Time	Source	Destination	Protocol	Length	Info
33	0.000000	192.168.5.128	192.168.5.2	DNS	97	Standard query 0xe50a A firefox.settings.services.mozilla.com

Specific IP addresses you identified in the traffic:

- 192.168.5.128 (eth0 ip) and 192.168.5.130 (owasp ip)
- 34.107.243.93, 34.107.221.82 (External/Public IPs)

Exercise 4:

- Select a packet and list the following information:

No.	Time	Source	Destination	Protocol	Length	Info
21	26.922395273	192.168.5.128	192.168.5.130	HTTP	789	POST /mutillidae/index.php?id=1 HTTP/1.1

- Source IP:192.168.5.128
- Destination IP: 192.168.5.130
- Protocol: HTTP
- Any TCP Flags observed: ACK(Acknowledge receipt of packets),FIN and ACK

Screenshot of TCP observerd.

No.	Time	Source	Destination	Protocol	Length	Info
5531	6896.7118405...	34.107.243.93	192.168.5.128	TLSv1.3	191	Application Data
5532	6896.7603956...	192.168.5.128	34.107.243.93	TCP	54	45810 → 443 [ACK] Seq=2070 Ack=1163 Win=63078 Len=0
5552	6944.5895634...	192.168.5.128	142.251.157.119	TLSv1.3	93	Application Data
5553	6944.5902746...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2762 Win=64240 Len=0
5554	6952.7267801...	192.168.5.128	142.251.157.119	TLSv1.3	93	Application Data
5555	6952.7271988...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2801 Win=64240 Len=0
5556	6952.7274135...	192.168.5.128	142.251.157.119	TLSv1.3	78	Application Data
5557	6952.7275450...	192.168.5.128	142.251.157.119	TCP	54	40302 → 443 [FIN, ACK] Seq=2825 Ack=1991 Win=62250 Len=0
5558	6952.7278351...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2825 Win=64240 Len=0
5559	6952.7278864...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [ACK] Seq=1991 Ack=2826 Win=64239 Len=0
5560	6953.7296496...	192.168.5.128	34.107.243.93	TLSv1.3	93	Application Data
5561	6953.7300407...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2752 Win=64240 Len=0
5562	6961.7697221...	192.168.5.128	34.107.243.93	TLSv1.3	93	Application Data
5563	6961.7701593...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2791 Win=64240 Len=0
5564	6961.7706317...	192.168.5.128	34.107.243.93	TLSv1.3	78	Application Data
5565	6961.7708350...	192.168.5.128	34.107.243.93	TCP	54	45804 → 443 [FIN, ACK] Seq=2815 Ack=862 Win=63379 Len=0
5566	6961.7709510...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2815 Win=64240 Len=0
5567	6961.7721550...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [ACK] Seq=862 Ack=2816 Win=64239 Len=0
5570	6985.2704599...	142.251.157.119	192.168.5.128	TCP	60	443 → 40302 [RST, ACK] Seq=1991 Ack=2826 Win=64239 Len=0
5571	6988.0613536...	34.107.243.93	192.168.5.128	TCP	60	443 → 45804 [RST, ACK] Seq=862 Ack=2816 Win=64239 Len=0
5676	7196.7952104...	34.107.243.93	192.168.5.128	TLSv1.3	78	Application Data
5677	7196.7952412...	192.168.5.128	34.107.243.93	TCP	54	45810 → 443 [ACK] Seq=2070 Ack=1187 Win=63054 Len=0
5678	7196.7954469...	192.168.5.128	34.107.243.93	TLSv1.3	82	Application Data
5679	7196.7958428...	34.107.243.93	192.168.5.128	TCP	60	443 → 45810 [ACK] Seq=1187 Ack=2098 Win=64240 Len=0

Exercise 5:

- Follow a TCP stream for a specific session and summarize the data exchanged between the client and server.

Wireshark - Packet 2662 - eth0

tcp.stream eq 49

Frame 2662: Packet, 1294 bytes on wire (10352 bits), 1294 bytes captured (10352 bits) on interface eth0, id 0

Section number: 1

Interface id: 0 (eth0)

Encapsulation type: Ethernet (1)

Arrival Time: May 23, 2026 11:43:33.465097753 EDT

UTC Arrival Time: May 23, 2026 15:43:33.465097753 UTC

Epoch Arrival Time: 1779551013.465097753

[Time shift for this packet: 0.00000000 seconds]

[Time delta from previous captured frame: -163.296 microseconds]

[Time delta from previous displayed frame: 38.516561 milliseconds]

[Time since reference or first frame: 53 minutes, 43.032334917 seconds]

Frame Number: 2662

Frame Length: 1294 bytes (10352 bits)

Capture Length: 1294 bytes (10352 bits)

[Frame is marked: False]

0000 00 0c 29 4b 12 cf 00 50 56 e2 35 4d 08 00 45 00 ...K...P V 5M...E

0010 05 00 49 8f 00 00 80 11 fe c2 8e fb 98 77 c0 a8 ...I...w...

0020 05 80 01 bb d3 2a 04 ec 49 84 cc 00 00 00 01 03 ...*...I...

0030 36 97 5c 08 ee 6a ea a0 d7 98 21 f5 00 40 20 0e 6.\...j...!...@...

0040 e8 45 87 e5 5c 4f c5 bb 05 63 ac 9d 65 5f fc f1 E\O...c...e...

0050 de 11 01 b6 11 80 bf 98 64 e6 e4 a3 06 46 1e e4 ...d...F...

0060 00 00 00 01 03 36 97 5c 08 ee 6a ea a0 d7 98 21 ...6.\...j...!

0070 f5 44 9b 51 0c b6 2e a4 ab 08 e6 2f e0 70 d6 63 D.Q...../.p.c...

0080 08 0e 52 c8 0f f6 04 77 70 77 d4 fd 25 57 99 be ..R...w pw...%W...
 0090 eb 2d c8 61 3d 6f b8 9a 7e d1 ac 60 85 cf 3c 49 ...a-o... ..<I...
 00a0 83 c3 e1 ad e8 7a d6 c1 f8 7e ed fb 19 d0 f7 9f ...z... ..
 00b0 e8 2e 97 d6 95 45 28 48 2c 4f 7e 32 21 62 10 17 ...E(H ,0-2!b...
 00c0 59 50 65 52 bf 35 66 7c a6 14 c2 09 1f ab 4c 5a YPeR 5f]LZ

1. The analyzed TCP stream (tcp.stream eq 49) showed active communication between the local host 192.168.5.128 and an external/public internet server 142.125.152.119 using Internet Protocol Version 4 (IPv4).
2. The capture used Ethernet II encapsulation on the eth0 network interface. The encapsulation type was identified as Ethernet (1).
3. The stream contained TCP packets with relatively large payload sizes of up to 1294 bytes. The frame length was also 1294 bytes, indicating the transfer of application data rather than simple acknowledgment packets(ACK).
4. The packet contents appeared unreadable and randomized, suggesting that the communication was encrypted using TLS/HTTPS protocols. The Time To Live (TTL) value observed was 128.
5. Timing analysis showed a fast response interval of approximately 38 ms between packets, indicating stable and responsive network communication.

Exercise 6:

- Take a screenshot of the Protocol Hierarchy and analyze the data.

Wireshark - Protocol Hierarchy Statistics - eth0

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	Bits/s	End Packets	End Bytes	End Bits/s	PDU's
Frame	100.0	9338	100.0	1641818	1,153	0	0	0	9338
Ethernet	100.0	9338	11.4	187056	131	0	0	0	9338
Internet Protocol Version 6	9.8	911	2.4	38736	27	0	0	0	911
User Datagram Protocol	6.2	575	0.3	4600	3	0	0	0	575
Multicast Domain Name System	4.2	390	0.9	15568	10	390	15568	10	390
Link-local Multicast Name Resolution	2.0	185	0.4	6308	4	185	6308	4	185
Internet Control Message Protocol v6	3.6	336	0.5	8648	6	336	8648	6	336
Internet Protocol Version 4	58.0	5417	6.7	109360	76	0	0	0	5417
User Datagram Protocol	25.3	2360	1.1	18880	13	0	0	0	2360
Simple Service Discovery Protocol	1.1	107	0.9	14659	10	107	14659	10	107
QUIC IETF	4.5	423	17.1	280203	196	422	231521	162	546
Malformed Packet	0.0	1	0.0	0	0	1	0	0	1
Network Time Protocol	6.3	586	1.7	28128	19	586	28128	19	586
NetBIOS Name Service	0.1	12	0.0	600	0	12	600	0	12
NetBIOS Datagram Service	0.3	32	0.2	2624	1	0	0	0	32
SMB (Server Message Block Protocol)	0.3	32	0.3	4432	3	0	0	0	32
SMB MailSlot Protocol	0.3	32	0.0	800	0	0	0	0	32
Microsoft Windows Browser Protocol	0.3	32	0.1	1680	1	32	1680	1	32
Multicast Domain Name System	4.2	390	0.9	15568	10	390	15568	10	390
Link-local Multicast Name Resolution	2.0	185	0.4	6308	4	185	6308	4	185
Dynamic Host Configuration Protocol	1.0	94	1.7	28152	19	94	28152	19	94
Domain Name System	5.7	531	2.3	37000	25	531	37000	25	531
Transmission Control Protocol	28.8	2693	3.6	58664	41	1764	40012	28	2693
Transport Layer Security	9.7	907	40.8	669390	470	907	629582	442	915
Hypertext Transfer Protocol	0.2	22	0.5	8594	6	9	3011	2	22
Media Type	0.0	1	0.1	1150	0	1	1150	0	1
Line-based text data	0.1	10	6.1	100732	70	10	100732	70	10
HTML Form URL Encoded	0.0	2	0.0	121	0	2	121	0	2
Internet Group Management Protocol	2.7	255	0.3	4120	2	255	4120	2	255
Internet Control Message Protocol	1.2	109	0.6	10161	7	109	10161	7	109

No display filter.

Close Copy Protocols Help

- Which protocol is most prevalent in your capture?

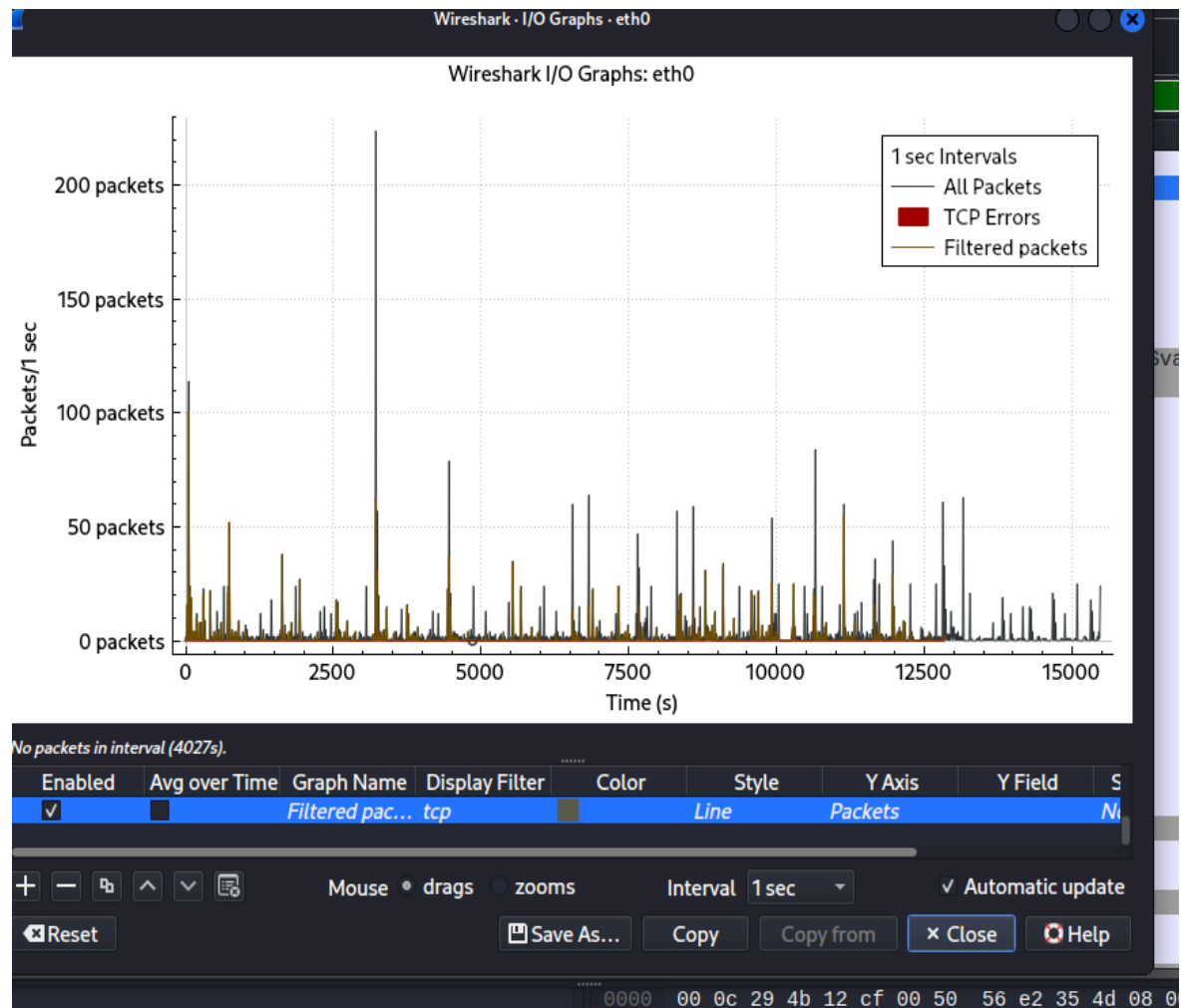
The Protocol Hierarchy Statistics showed that the Transmission Control Protocol (TCP) was the most prevalent protocol in network capture, accounting for 28.8% of all packets (2693 packets).

- IO Graphs:
 - Access Statistics > IO Graphs to visualize traffic over time.

Exercise 7:

- Create an IO Graph showing TCP traffic. Describe any noticeable patterns you observe:

Screenshot of IO Graphs:



Noticeable Patterns Observed:

The IO Graph showed a smooth and continuous flow of network traffic over time, indicating stable and normal communication between hosts.

Several sudden sharp upward peaks were also observed in the graph. These traffic spikes suggest periods of increased network activity and high packet transmission, likely caused by encrypted TLS/HTTPS communication, DNS query during internet access .

Exercise 8:

- Save your capture file and describe a scenario where you would need to review this data later. What specific findings do you hope to extract?

A situation where this data may need to be reviewed later is during the investigation of suspicious network behavior, slow internet performance, or unauthorized communication with external servers.

From the captured traffic, I would hope to extract:

- Source and destination IP addresses involved in communication,
- TCP sessions and connection details,
- HTTP and TLS/HTTPS activity,
- DNS requests and responses,
- Packet timing and response delays,
- Indicators of suspicious or malicious traffic.

Exercise 9:

- Describe a real-world scenario where you would use Wireshark to troubleshoot a network issue. What specific symptoms would you investigate?
-
- A real-world scenario for using Wireshark would be troubleshooting slow or unstable internet connectivity in an organization or home

network. For example, users may complain that websites load slowly or online applications frequently disconnect.

Using Wireshark, the investigation would focus on symptoms such as:

- High packet loss,
- Excessive TCP retransmissions,
- DNS resolution failures,
- Delayed server responses,
- Connection resets (RST packets),
- High latency between client and server,
- Unusual traffic spikes or bandwidth usage.

Wireshark would help identify whether the issue is caused by the local device, router, ISP, or external servers.

Exercise 10:

Identify at least two potential security threats in your captured traffic.
What indicators led you to suspect these activities?

Two potential security threats observed in the captured traffic include:

1. Unencrypted HTTP Traffic

The capture contained HTTP packets such as:

- GET /success.txt?ipv4 HTTP/1.1

This is a potential security risk because HTTP traffic is transmitted in plain text and may expose sensitive information to attackers performing packet sniffing or man-in-the-middle attacks.

2. Communication with External Public Servers

The system communicated with external public IP addresses such as:

- 34.107.243.93
- 34.107.221.82
- 142.125.152.119

Frequent encrypted communication with unknown external servers may indicate:

- background application activity.
- cloud service connections
- or potentially suspicious outbound traffic.

The indicators that raised attention included:

- multiple TLS sessions,
- repeated TCP handshakes.
- continuous data exchange with external hosts,
- and high numbers of ACK and PSH/ACK packets.

Although no direct malicious activity was confirmed, these patterns would require further investigation in a real security analysis.

INT302: Kali Linux Tools and System Security – Lab 6: Advanced Packet Analysis Techniques

Exercise 1:

- Describe the purpose of the SYN and ACK flags in the TCP handshake. How do these flags indicate the status of a connection?

No.	Time	Source	Destination	Protocol	Length	Info
517	94.888364928	192.168.5.130	192.168.5.128	TCP	74	80 → 40158 [SYN, ACK] Seq=0 Ack=1 Win=5792 Len=0 MSS=1460 SACK_PERM TSval=892791 TSecr=3119287179 WS=64
518	94.888444496	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=1 Ack=1 Win=64256 Len=0 TSval=3119287179 TSecr=892791
519	94.888629267	192.168.5.128	192.168.5.130	HTTP	792	POST /mutillidae/index.php?page=login.php HTTP/1.1 (application/x-www-form-urlencoded)
520	94.889832613	192.168.5.130	192.168.5.128	TCP	66	80 → 40158 [ACK] Seq=1 Ack=727 Win=7296 Len=0 TSval=892791 TSecr=3119287179
521	94.907040652	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=1 Ack=727 Win=7296 Len=1448 TSval=892795 TSecr=3119287179 [TCP PDU reassembled in 52]
522	94.907081564	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=1449 Win=67328 Len=0 TSval=3119287198 TSecr=892795
523	94.907167912	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=1449 Ack=727 Win=7296 Len=1448 TSval=892796 TSecr=3119287179 [TCP PDU reassembled in 52]
524	94.907173885	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=2897 Win=78144 Len=0 TSval=3119287198 TSecr=892796
525	94.908085274	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=2897 Ack=727 Win=7296 Len=1448 TSval=892796 TSecr=3119287179 [TCP PDU reassembled in 52]
526	94.908083253	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=4345 Win=72960 Len=0 TSval=3119287199 TSecr=892796
527	94.908444553	192.168.5.130	192.168.5.128	HTTP	5123	HTTP/1.1 200 OK (text/html)
528	94.908460792	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [ACK] Seq=727 Ack=9402 Win=80640 Len=0 TSval=3119287199 TSecr=892796
529	94.916417171	192.168.5.128	192.168.5.130	HTTP	597	GET /mutillidae/images/favicon.ico HTTP/1.1
530	94.976646182	192.168.5.128	192.168.5.130	TCP	66	40158 → 80 [FIN, ACK] Seq=1258 Ack=9402 Win=80640 Len=0 TSval=3119287267 TSecr=892796
531	94.977934824	192.168.5.130	192.168.5.128	TCP	1514	80 → 40158 [ACK] Seq=9402 Ack=1259 Win=8784 Len=1448 TSval=892813 TSecr=3119287267 [TCP PDU reassembled in 53]
532	94.977960745	192.168.5.128	192.168.5.130	TCP	54	40158 → 80 [RST] Seq=1259 Win=0 Len=0
533	94.978055024	192.168.5.130	192.168.5.128	HTTP	249	HTTP/1.1 200 OK (Image/x-icon)
534	94.978061319	192.168.5.128	192.168.5.130	TCP	54	40158 → 80 [RST] Seq=1259 Win=0 Len=0
537	107.195584288	192.168.5.128	142.251.152.119	TLSv1.3	93	Application Data
538	107.196090190	142.251.152.119	192.168.5.128	TCP	60	443 → 51164 [ACK] Seq=6434 Ack=19512 Win=64240 Len=0
539	107.237855186	142.251.152.119	192.168.5.128	TLSv1.3	93	Application Data
540	107.272684431	192.168.5.128	142.251.152.119	TCP	54	51164 → 443 [ACK] Seq=19512 Ack=6473 Win=61320 Len=0
543	120.283361576	192.168.5.128	142.251.152.119	TLSv1.3	93	Application Data
544	120.294391148	142.251.152.119	192.168.5.128	TCP	60	443 → 34900 [ACK] Seq=5402 Ack=40000 Win=64240 Len=0
545	120.272669851	142.251.152.119	192.168.5.128	TLSv1.3	93	Application Data
546	120.312491804	192.168.5.128	142.251.152.119	TCP	54	34900 → 443 [ACK] Seq=40000 Ack=5441 Win=65535 Len=0
547	130.214348286	192.168.5.128	34.107.243.93	TLSv1.3	93	Application Data
548	130.214758210	34.107.243.93	192.168.5.128	TCP	60	443 → 43254 [ACK] Seq=862 Ack=2752 Win=64240 Len=0

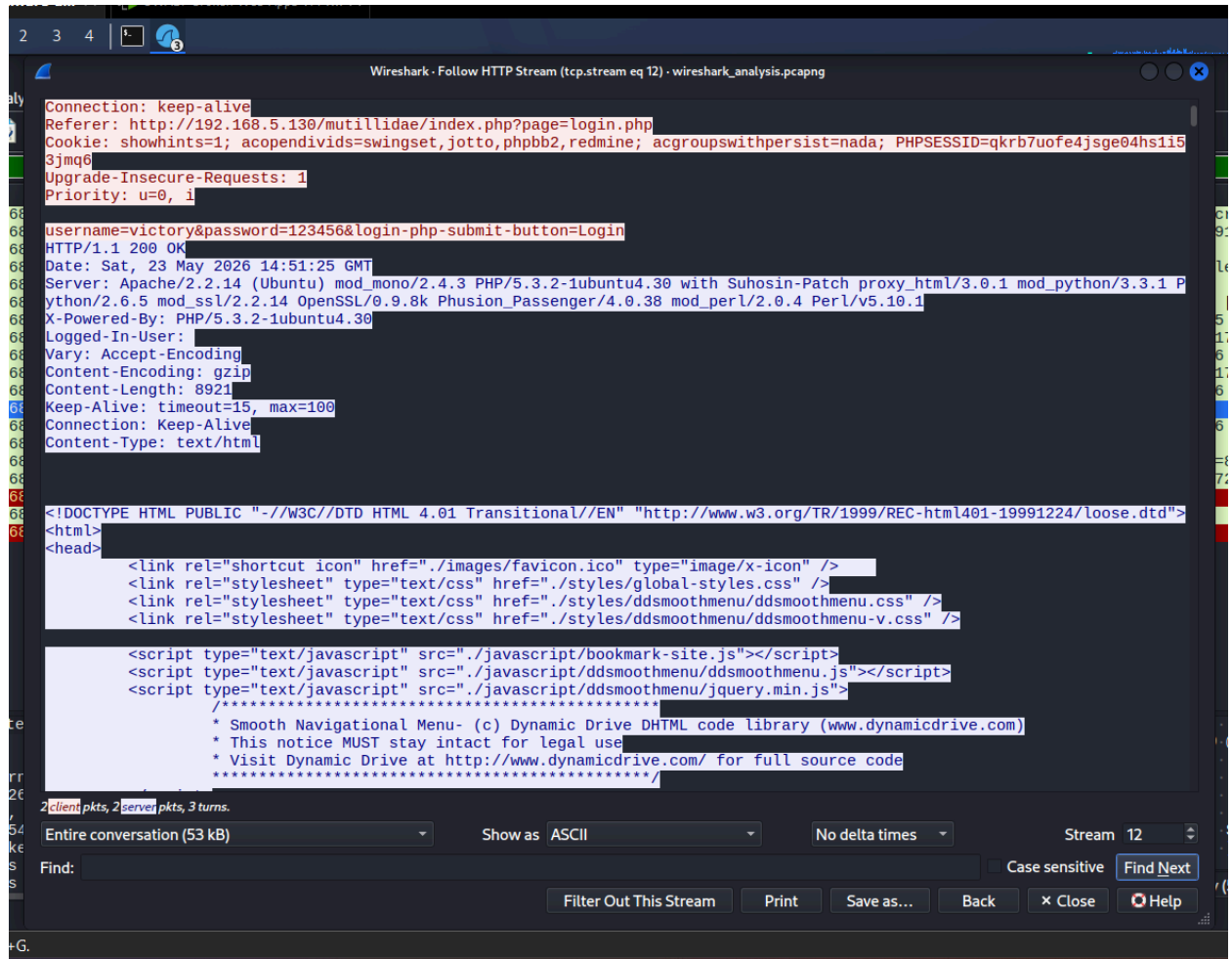
SYN and ACK flags indicated the establishment and acknowledgment of a TCP connection between the client (192.168.5.128) and the external/public server.

- The SYN (Synchronize) flag was used by the client to initiate the TCP connection with the server.
- The ACK (Acknowledgment) flag confirmed the receipt of packets between both communicating hosts.

Exercise 2:

- Choose an HTTP packet and summarize its request method, status code, and any notable headers. What can you infer about the transaction? _____

Screenshot of HTTP packet analyzed.



The request method used was POST /mutillidae/index.php?page=login.php
HTTP/1.1

The server responded with the status code: HTTP/1.1 200 OK

Several notable headers were observed, including:

- Content-Type: application/x-www-form-urlencoded
- Cookie: PHPSESSID=...
- User-Agent: Mozilla Firefox

- Server: Apache/2.2.14 (Ubuntu)
- Connection: Keep-Alive

The packet also contained login credentials transmitted in plain text:

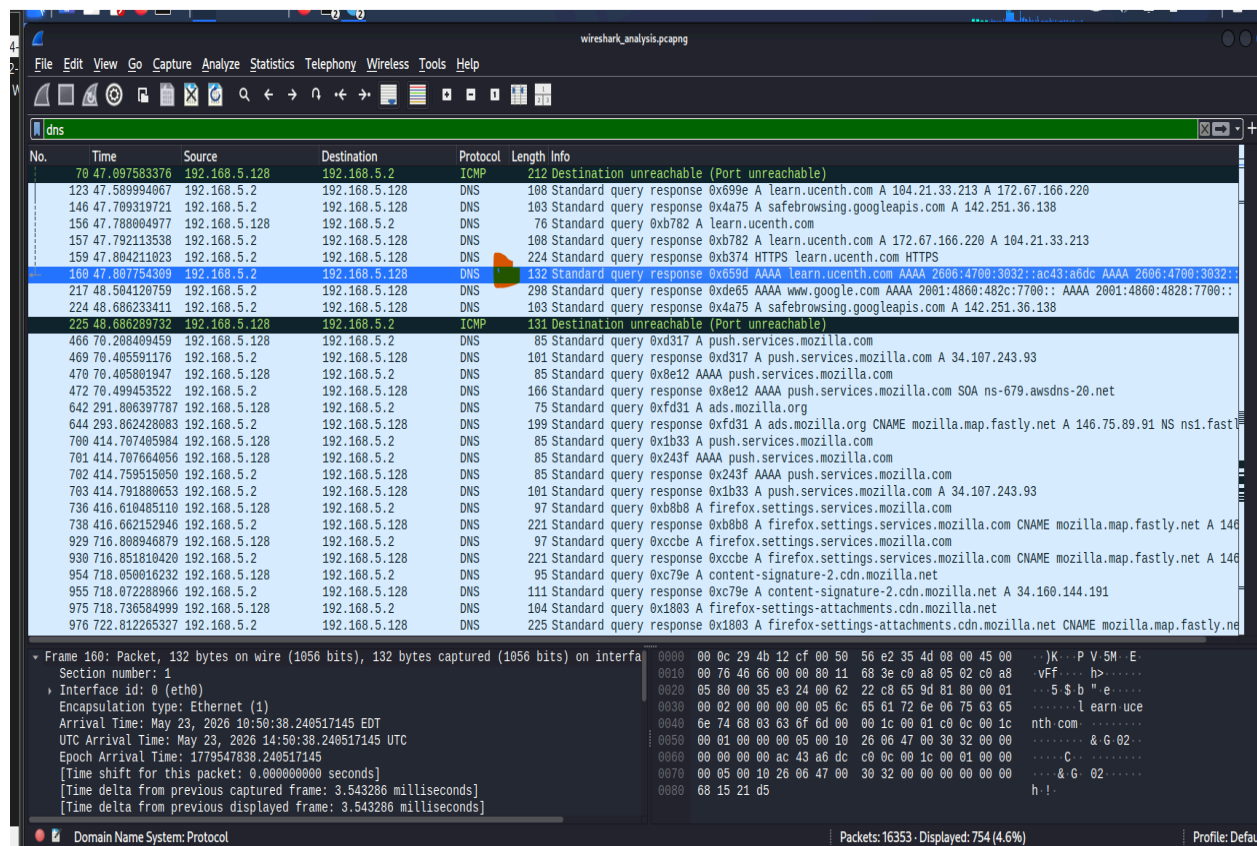
username=victory&password=123456

From this transaction, it can be inferred that the user attempted to authenticate to the Mutillidae web application using HTTP rather than HTTPS. This means sensitive information, including usernames, passwords, and session cookies, was exposed in clear text and could be intercepted by attackers monitoring the network traffic.

Exercise 3:

- Identify a DNS query and its corresponding response. What information does the response provide, and how is it structured?

Screenshot of Identified DNS query and response.



The DNS packet shown is a DNS response from 192.168.5.2 to 192.168.5.128.

The response provides:

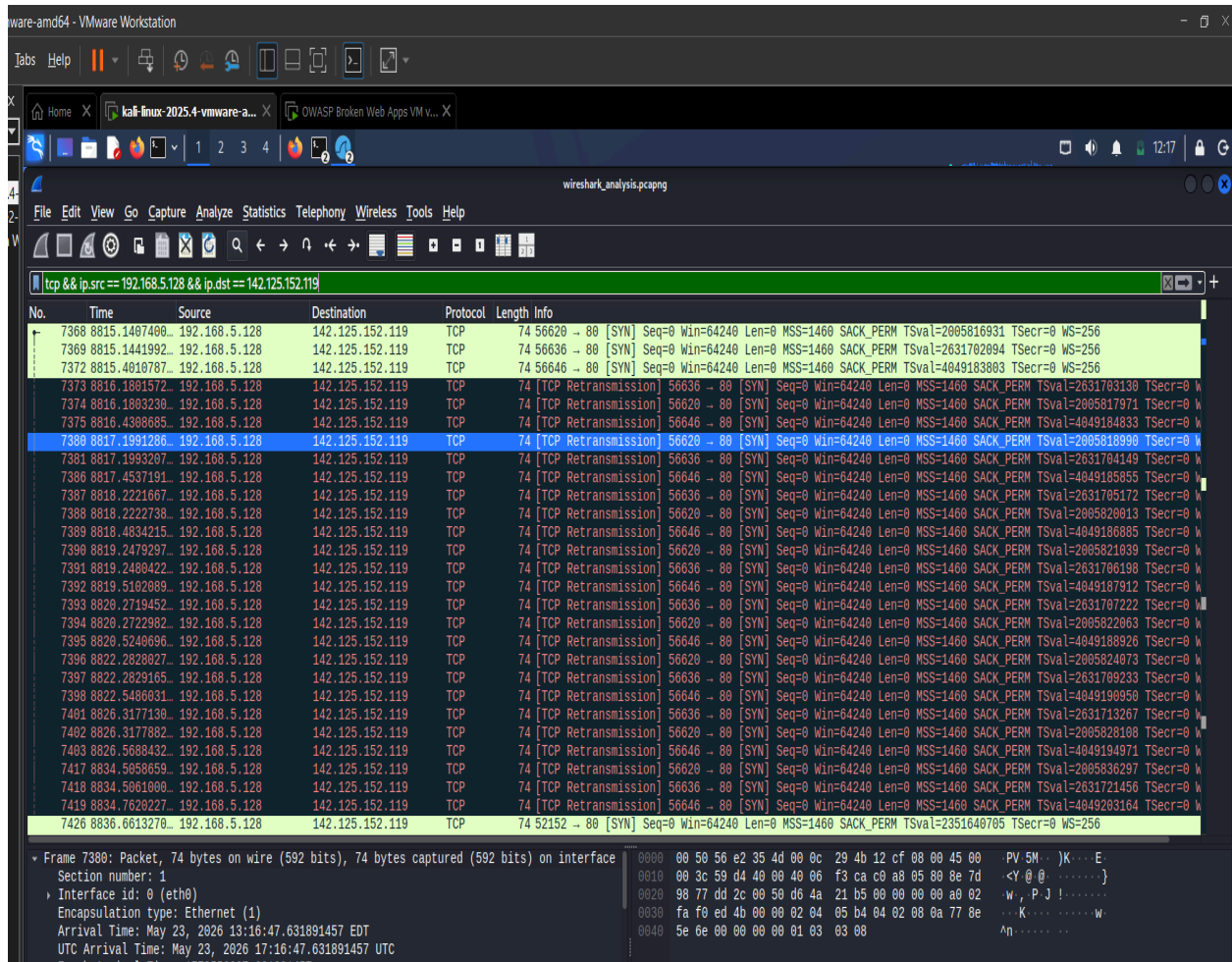
- the requested domain name (learn.ucenth.com),
- the query type (AAAA record),
- and the resolved IPv6 addresses:
2606:4700:3032::ac43:a6dc
2606:4700:3032::6815:21d5

This means the client requested the IPv6 address of the domain, and the DNS server responded successfully with two valid IPv6 addresses associated with that hostname.

Exercise 4:

- Custom filter that captures only TCP traffic from my machine to a specific target IP. Document the filter syntax and the packets captured. _____

Screenshot of custom filter that captures only TCP traffic



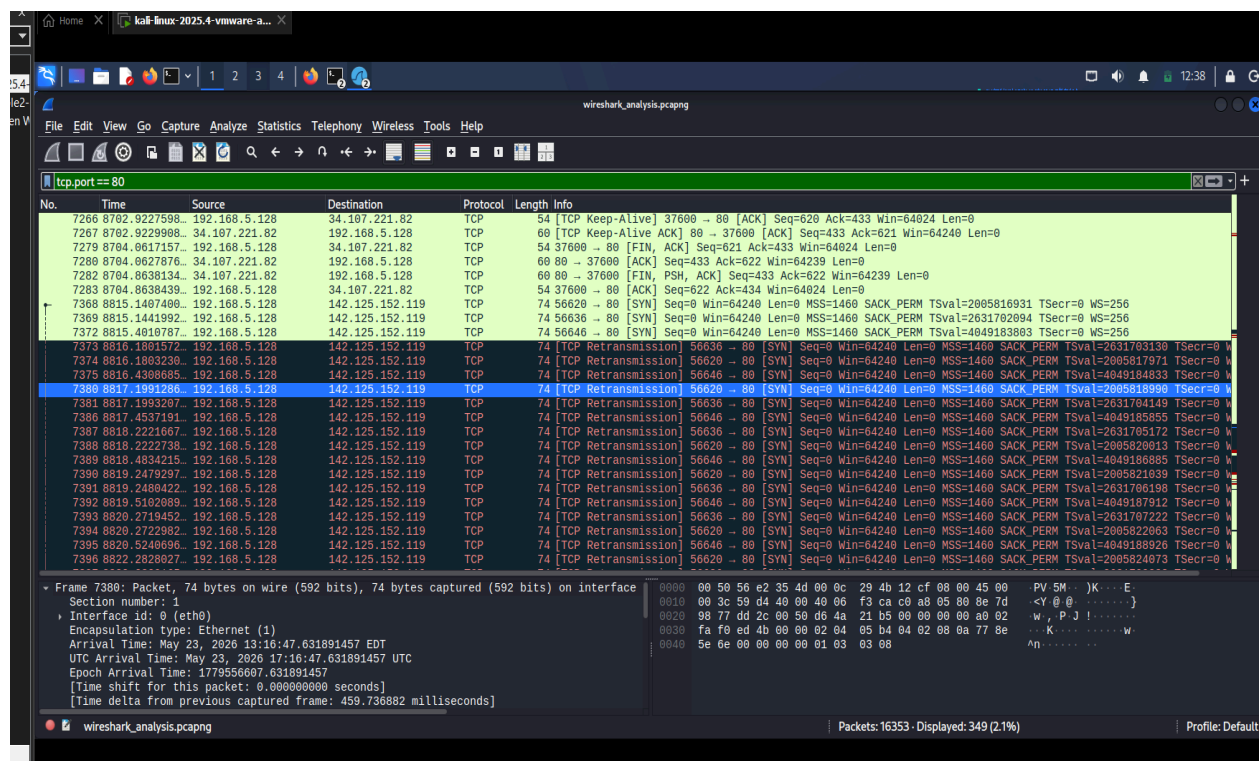
Custom Filter Syntax

`tcp && ip.src == 192.168.5.128 && ip.dst == 142.125.152.119`

The packets captured in this filter are primarily TCP SYN packets and TCP Retransmission SYN packets sent from client machine (192.168.5.128) to the target IP 142.125.152.119 on port 80 (HTTP).

Exercise 5:

- filter that captures traffic on a specific port (e.g., HTTP port 80) and analyzes the results. What packets were captured?



Filter Syntax

tcp.port == 80

The packets captured in this filter are TCP[Keep - Alive,ACK], TCP[FIN,PSH,ACK], TCP SYN packets and TCP Retransmission SYN

packets sent from client machine (192.168.5.128) to the target IP 142.125.152.119 on port 80 (HTTP).

Exercise 6:

- Analyze your capture for any anomalies or indicators of potential vulnerabilities. Document your findings and suggest possible remediation steps. _____

Findings & Traffic Behavior Analysis

1. SYN and SYN Retransmissions

What this indicates:

- The client is attempting to establish a TCP connection (3-way handshake).
- Retransmitted SYN packets indicated that the server did not respond to the initial SYN.

Possible causes:

- Packet loss on the network
- Firewall or security device silently dropping packets
- Target port is filtered or intermittently unreachable
- Server overload or rate-limiting SYN requests

Security implication:

- This can resemble a port scan or probing behavior
- Could indicate a filtered or protected HTTP service

2. TCP Keep-Alive ACK

What this means:

- A previously established connection is being maintained.

- The system is checking whether the remote host is still responsive.

Implications:

- Suggests at least some connections were successfully established earlier.
- Could also indicate idle connection monitoring by client or server.

3. TCP FIN, PSH, ACK

What this indicates:

- A connection is being closed properly (FIN handshake).
- PSH means data was actively pushed before closure.

Implication:

- At least one session successfully completed data exchange.
- Normal behavior for HTTP session termination.

Anomalies Detected

1. SYN Retransmissions (Key anomaly)

- Repeated SYN attempts without immediate response
- Suggests instability or filtering

This is the strongest indicator of a possible issue.

2. Mixed connection states:

- Failed connection attempts (SYN retries)
- Successful session behavior (FIN/PSH/ACK)
- Keep-alive traffic

This inconsistency may suggest:

- Intermittent network reachability
- Firewall rules blocking some sessions
- Load balancer or proxy behavior

3. Unencrypted HTTP traffic (port 80)

- All traffic is over HTTP, not HTTPS

Risk:

- Data can be intercepted or modified (MITM attacks)

Recommended Remediation Steps

1. Upgrade to HTTPS (Critical)

- Migrate service from HTTP (port 80) to HTTPS (port 443)
- Use TLS certificates (Let's Encrypt or enterprise CA)

2. Investigate SYN retransmissions such as:

- a. Firewall rules
- b. IDS/IPS logs
- c. Server availability and load

3. Enable SYN protection

On server/firewall:

- Enable SYN cookies
- Configure SYN flood protection

4. Apply rate limiting

- Prevent excessive connection attempts from same IP

5. Harden HTTP service (if still required)

- Disable unused HTTP methods (e.g., PUT, DELETE)
- Restrict access by IP if internal-only service

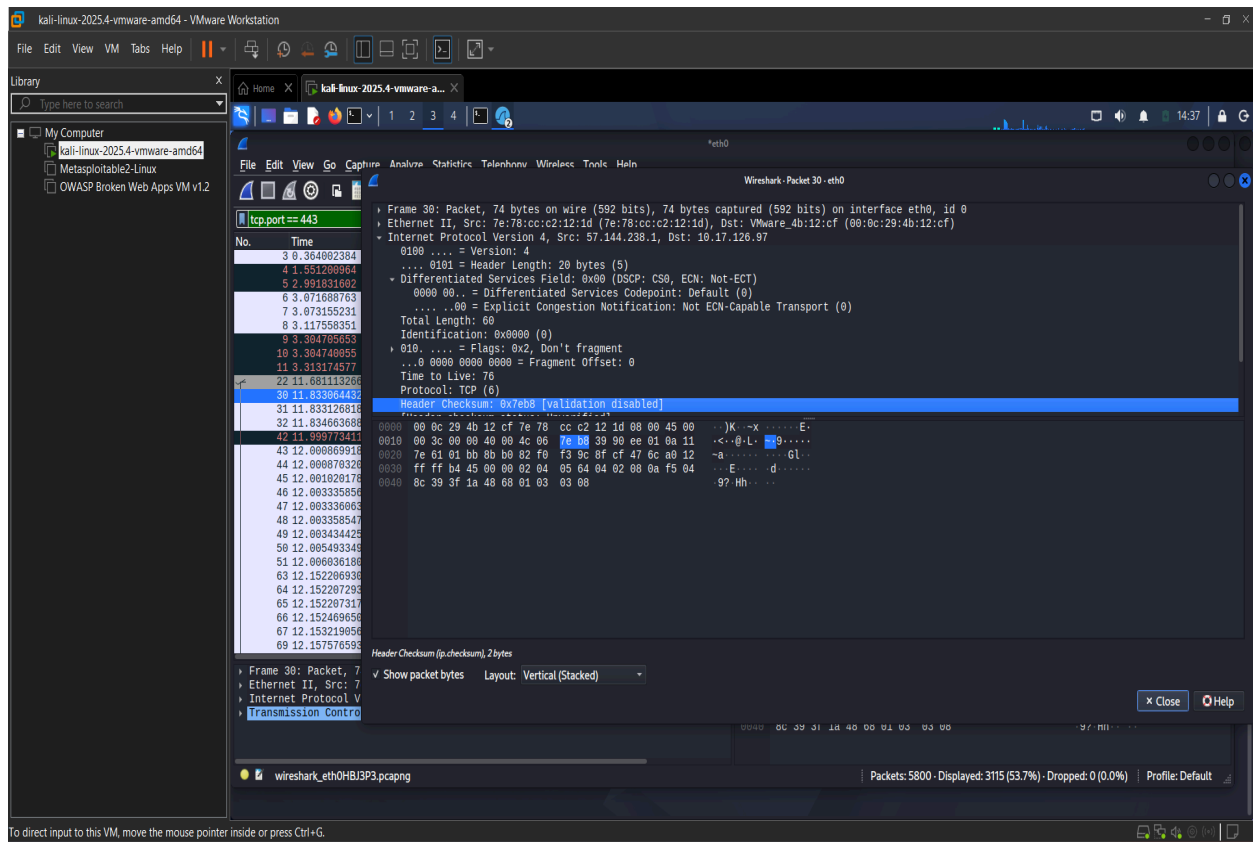
- Enable logging and monitoring

6. Monitor with IDS/IPS

- Use tools like:
 - a. Snort
 - b. Suricata
- Alert on:
 - a. SYN retransmission spikes
 - b. repeated failed handshakes

Exercise 7:

- Capture HTTPS traffic and identify the initial handshake packets. What information is exchanged during this handshake, and how does it contribute to security? _____



What information is exchanged during handshake

During HTTPS communication, Wireshark captures the TLS handshake which includes packets such as Client Hello, Server Hello, Certificate, Key Exchange, and Finished messages. In this process, the client and server exchange supported TLS versions, cipher suites, random values, and the server's digital certificate. The certificate contains the server's public key and verifies its identity through a trusted Certificate Authority. After this exchange, both parties derive a shared secret key using secure key exchange algorithms such as Diffie-Hellman.

How does it contribute to security

This handshake establishes an encrypted session, ensuring confidentiality, integrity, and authentication.

It protects against eavesdropping, data tampering, and man-in-the-middle attacks by ensuring that only the intended server can decrypt the communication and that transmitted data cannot be modified.

FINDINGS

- TCP 3-way handshake observed:
 SYN → SYN-ACK → ACK
 This confirms successful session establishment between systems.
- TCP flags identified:
 - SYN → connection initiation
 - SYN-ACK → server response
 - ACK → acknowledgment of communication
- Open ports discovered:
 - Port 22 → SSH (open)
 - Port 80 → HTTP (open)
 - Port 139 → NetBIOS/SMB (open)
- HTTP traffic:
 - HTTP packets were captured successfully
 - Traffic is unencrypted and readable
 - This creates a risk of data exposure during transmission
- HTTPS traffic:
 - HTTPS traffic could not be read due to encryption
 - Only TLS handshake was visible
 - Payload content was hidden
- Packet filtering:
 - Applying filters (e.g., HTTP) reduced visible packets
 - Removing filters restored full traffic

SECURITY ANALYSIS

Observed Weaknesses:

- Open SSH (22) may allow brute-force attacks
- HTTP traffic is unencrypted (MITM risk)
- SMB (139) can expose file sharing vulnerabilities
- Multiple exposed services increase attack surface
- Encrypted traffic reduces visibility for monitoring tools

RECOMMENDATIONS

- Replace HTTP with HTTPS (TLS encryption)
- Disable unused services (especially SMB/NetBIOS)
- Secure SSH with key-based authentication
- Apply firewall rules to restrict open ports
- Use IDS/IPS (e.g., Snort or Suricata)
- Regular monitoring and log analysis
- Perform periodic vulnerability assessments

Conclusion

The network analysis revealed multiple exposed services and insecure communication protocols within the Metasploitable environment. While the system is intentionally vulnerable for testing purposes, the findings demonstrate real-world risks such as open ports, unencrypted traffic, and potential attack vectors. Proper security controls such as encryption, service hardening, and monitoring are essential to mitigate these risks.